

H A S T H V A N I

Hardware Build & Wiring Manual

Two-glove ESP32 sign-language communication prototype

Left glove: 5 flex sensors + MPU6050 + ESP32 + TFT display

Right glove: 5 flex sensors + MPU6050 + ESP32 + MAX98357A + 3 W speaker

Mobile app: speech-to-text, wireless text to the left glove

Revision 1 · 26 September 2026

Source of truth: Hasth Vani Prototype Build Plan (With Directions) and (Illustrated) PDFs, plus the team's own bench measurements.

Where a module's exact pinout is not known, the manual says **VERIFY ON ACTUAL MODULE** instead of guessing.

Contents

- 1.** Project overview
- 2.** Component inventory
- 3.** System architecture
- 4.** Connection status register and conventions
- 5.** ESP32 pin map
- 6.** Single flex-sensor circuit
- 7.** Five-flex-sensor circuit
- 8.** Breadboard wiring: which holes are connected
- 9.** Previous wiring failure analysis
- 10.** MPU6050 wiring
- 11.** TFT wiring
- 12.** MAX98357A + speaker
- 13.** Power system
- 14.** Left glove complete circuit
- 15.** Right glove complete circuit
- 16.** Complete two-glove system
- 17.** Mobile app communication
- 18.** Gesture recognition
- 19.** Calibration
- 20.** Master flowchart: build sequence
- 21.** Master debugging decision tree
- 22.** Final assembly
- 23.** Wiring checklist and final test checklist
- 24.** Final validation report
- 25.** Deliverables and files

Figures and diagrams are numbered as in the deliverables list at the end. Diagrams are drawn as vector graphics: zoom in the PDF for detail.

1 Project overview

Hasht Vani is a two-glove prototype that lets a person who signs and a person who speaks communicate. Sign language is turned into voice by the **right glove**, and the partner's speech is turned into text on the **left glove**.

Part	What it does
Left glove (display)	5 flex sensors + MPU6050 → ESP32 LEFT → TFT. Receives text from the phone and shows it.
Right glove (speaker)	5 flex sensors + MPU6050 → ESP32 RIGHT (gesture recognition) → MAX98357A → 3 W speaker.
Mobile app	Microphone → speech-to-text → wireless text to the LEFT ESP32. Optional gesture history and language selection.

Main communication paths

SIGN LANGUAGE → RIGHT GLOVE → 5 FLEX + MPU6050 → ESP32 RIGHT → GESTURE RECOGNITION → MAX98357A → 3 W SPEAKER → VOICE
PARTNER SPEAKS → MOBILE APP → SPEECH TO TEXT → WIRELESS → ESP32 LEFT → TFT → TEXT

Where the project stands (from the progress report)

- ESP32 works; sketches compile and upload on **COM7** (Arduino IDE 2.3.10, board *ESP32 Dev Module*, 115200 baud Serial Monitor).
- The first five-flex breadboard test read about 0–150 on every channel and bending one sensor appeared to change the others. A 10 kΩ resistor swapped in for a sensor also read about 0. Averaging and delays did not help.
- **Conclusion:** 3.3 V was not reaching the sensor rows, so it is a wiring/contact fault, not a code fault and not a series-versus-parallel design problem. Sections 8 and 9 rebuild the wiring one finger at a time and give a multimeter procedure to prove the supply first.

How to read this manual

Every wire is drawn in a colour *and* labelled with text. Connection status is marked three ways: **SOURCE** (taken from the Hasht Vani PDFs), **SUGGESTED** (added in this manual) and **VERIFY** (must be checked against your actual module before permanent wiring).

2 Component inventory

Component	Qty	Use / note
ESP32 development board	2	Both gloves (LEFT / RIGHT); 30-pin DevKit with USB-C in the prototype
Flex sensor	10	5 per glove; ~25 k Ω flat (source PDF)
MPU6050	2	1 per glove; I2C
TFT display 2.4"/2.8"	1	Left glove; ILI9341 SPI typical — verify pinout
MAX98357A I2S amplifier	1	Right glove
Speaker 3 W (4–8 Ω)	1	Right glove — NOT yet in hand
3.7 V Li-ion battery	2	1 per glove
TP4056 charging module	2	1 per glove; use the version with protection
10 k Ω resistor	10	5 per glove: the fixed half of each divider
Switch	2	1 per glove, main power
Gloves	2	left + right
Jumper wires	—	as required
Perfboard / PCB	2	1 per glove
Breadboards	—	prototyping only

Not in the inventory but needed (VERIFY / obtain)

- **Digital multimeter** — required for the troubleshooting procedure (Section 9).
- **5 V boost module** (MT3608 or similar) — recommended by the source PDF because a single 3.7 V cell is below what many ESP32 boards' VIN regulators need. Confirm your board's VIN range first.
- A **USB cable that carries data** (not charge-only), heat-shrink or Kapton, hot glue, velcro, label tape.

VERIFY: which ESP32 board layout you have. This manual uses the 30-pin DevKit header order confirmed from the team's board photo (left row: 3V3, GND, D15, D2, D4, D16, D17, D5, D18, D19, D21, RX0, TX0, D22, D23; right row: VIN, GND, D13, D12, D14, D27, D26, D25, D33, D32, D35, D34, VN, VP, EN, read with USB-C at the top). Printed labels differ between vendors.

3 System architecture

DIAGRAM 1: HASTH VANI COMPLETE SYSTEM ARCHITECTURE

Two independent gloves. They are connected only wirelessly through the mobile app, never by wires.

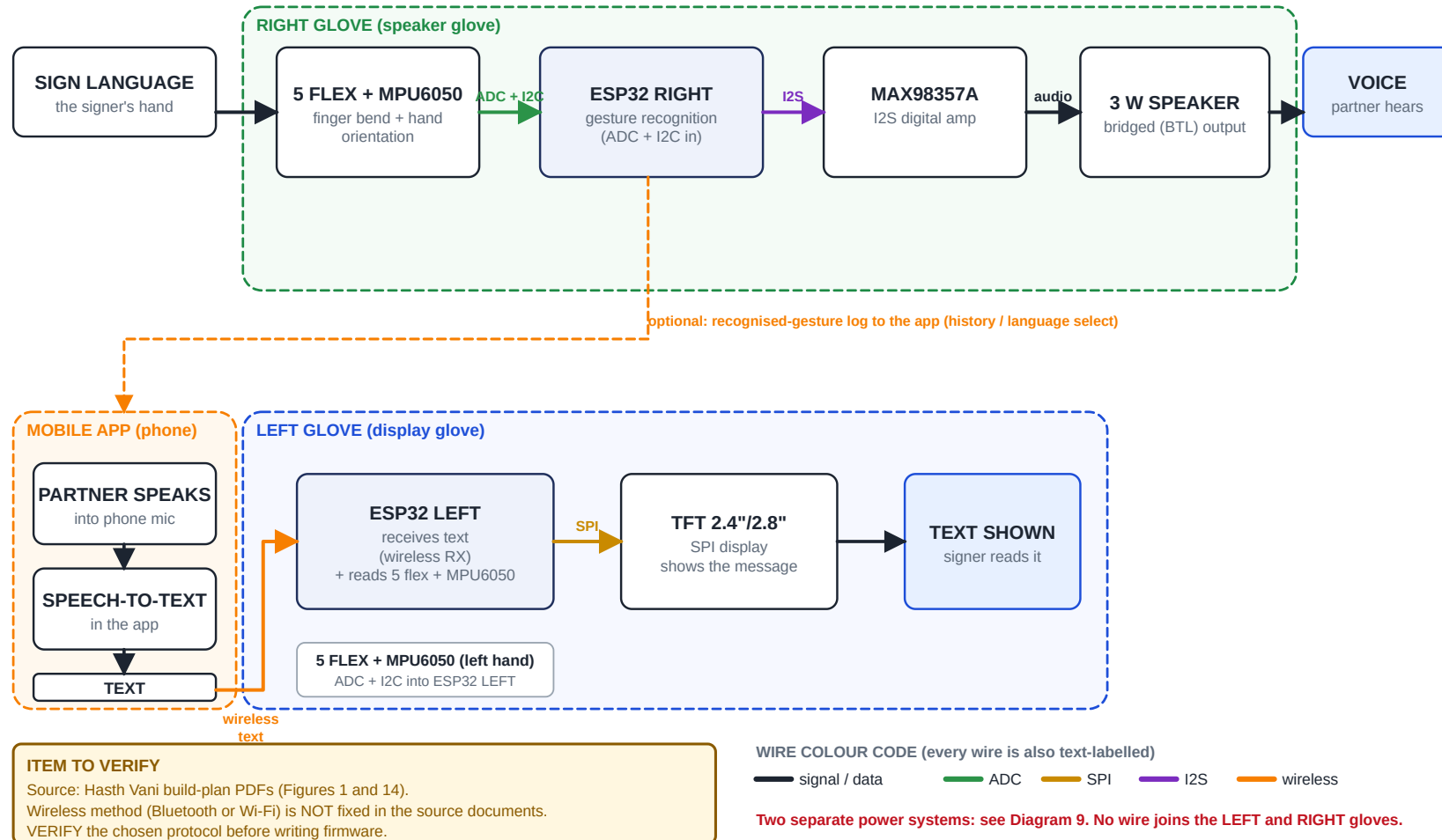


Diagram 1. The two gloves are independent; only the phone link connects them.

4 Connection status register and conventions

Connection	Status	Action
Flex sensors to ESP32 ADC1: GPIO36 (VP), 39 (VN), 34, 35, 32	SOURCE (PDF pin map, Fig. 5)	
Voltage divider: 3V3 → flex → node → 10 kΩ → GND	SOURCE (steps 6–8, Fig. 4)	
MPU6050: 3V3, GND, SDA GPIO21, SCL GPIO22, AD0 to GND, INT unused	SOURCE (Fig. 8)	Verify pin order and pull-ups on your module
TFT: CS15, RST4, DC2, MOSI23, SCK18, MISO19, VCC 3V3, LED 3.3 V	SOURCE (Fig. 10)	VERIFY THE TFT MODULE PINOUT; GPIO2/15 are strapping pins
MAX98357A: BCLK26, LRC25, DIN27, VIN from 5 V, GAIN open, SD open	SOURCE (Fig. 12)	Verify pin labels on the actual amplifier board
Speaker to amp OUT + / OUT - only, never to GND	SOURCE (Fig. 12 note)	Safety rule
Power: battery → TP4056 → switch → 5 V boost → ESP32 VIN; 3V3 to sensors/TFT	SOURCE (Fig. 15); boost is "recommended add-on"	VERIFY EXACT MODULE POWER REQUIREMENTS BEFORE CONNECTION
Breadboard test rows, jumper hole positions (Diagrams 13–16)	SUGGESTED (this manual)	Follows the hole tables exactly; checked by a netlist program
D33 (GPIO33, HIGH) as the 3.3 V source on the single breadboard only	SUGGESTED workaround	Only because the 3V3 pin sits under the ESP32 body; use the real 3V3 pin on the glove perfboard
Wireless protocol (Bluetooth or Wi-Fi), packet format	NOT FIXED in the source	VERIFY and record your choice
Gesture thresholds and gesture list	EXAMPLE only in the source	Must come from your own measured samples

Wire colour convention (colour is never the only cue: every wire has a text label)

 RED	VCC / positive supply	 BLACK	GND
 BLUE	I2C (SDA / SCL)	 GREEN	ADC / sensor signal
 YELLOW	SPI (TFT)	 PURPLE	I2S (audio)
 ORANGE	wireless / data / control		

5 ESP32 pin map

Reference mapping from the source PDF. **These are reference mappings: verify the physical module labels before permanent wiring.**

Flex sensors (both ESP32 boards, identical)

Finger	GPIO	Printed label	Type	Status
Thumb (T)	GPIO36	VP	ADC1	SOURCE
Index (I)	GPIO39	VN	ADC1	SOURCE
Middle (M)	GPIO34	D34	ADC1	SOURCE
Ring (R)	GPIO35	D35	ADC1	SOURCE
Little (L)	GPIO32	D32	ADC1	SOURCE

MPU6050 (both boards)

MPU6050 pin	ESP32	Status
VCC	3V3	SOURCE (verify module voltage)
GND	GND	SOURCE
SDA	GPIO21	SOURCE
SCL	GPIO22	SOURCE
AD0	GND (address 0x68)	SOURCE

TFT (LEFT board only)

TFT pin	ESP32	Status
CS	GPIO15	SOURCE, VERIFY (strapping pin)
RESET	GPIO4	SOURCE, VERIFY
DC / RS	GPIO2	SOURCE, VERIFY (strapping pin)
SDI (MOSI)	GPIO23	SOURCE, VERIFY
SCK	GPIO18	SOURCE, VERIFY
SDO (MISO)	GPIO19	SOURCE (only for touch / read-back)
VCC	3V3	SOURCE, VERIFY
GND	GND	SOURCE
LED (backlight)	3.3 V (or via 100 Ω / transistor)	SOURCE, VERIFY ON ACTUAL MODULE

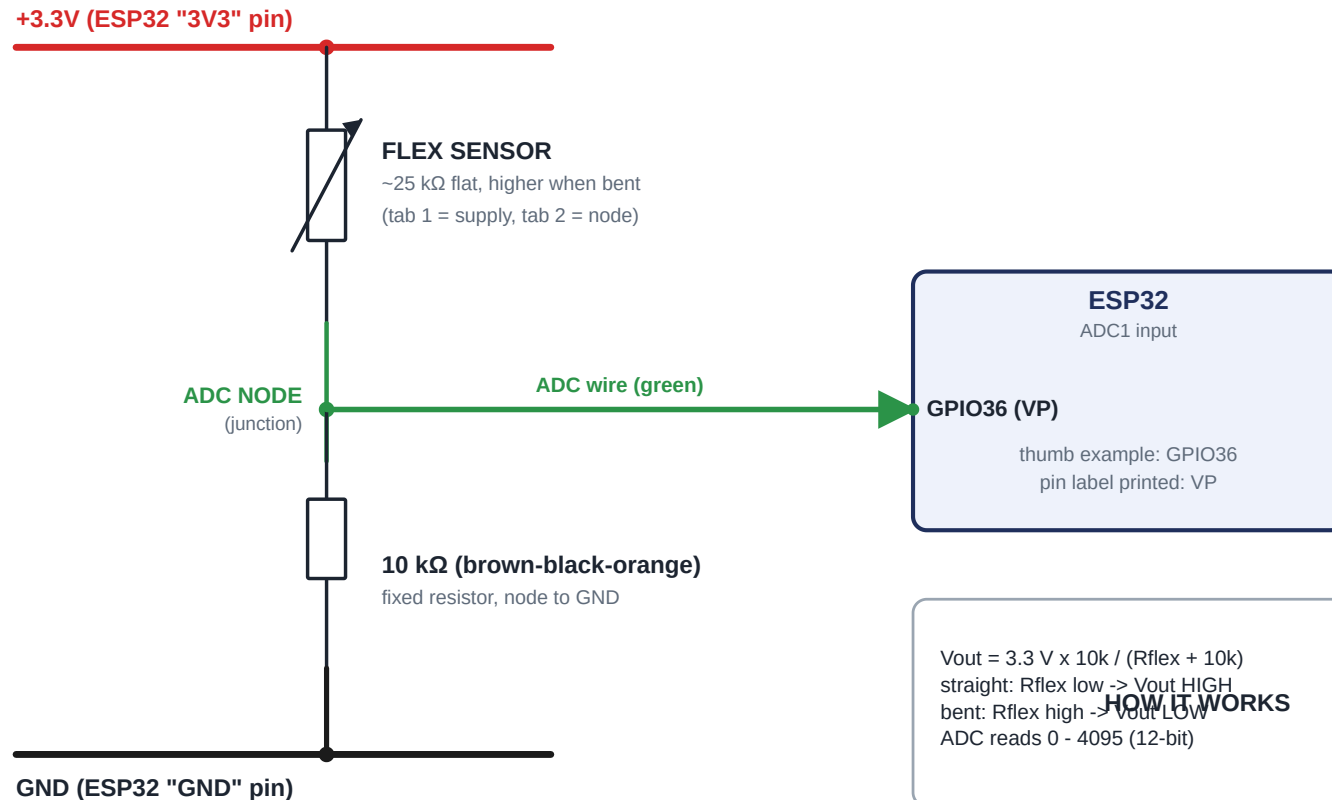
MAX98357A (RIGHT board only)

Amp pin	ESP32 / supply	Status
BCLK	GPIO26	SOURCE, VERIFY label

Amp pin	ESP32 / supply	Status
LRC / WS	GPIO25	SOURCE, VERIFY label
DIN	GPIO27	SOURCE, VERIFY label
VIN	5 V / VBOOST	SOURCE, VERIFY module supply range
GND	GND	SOURCE
GAIN / SD	left open (9 dB / on)	SOURCE
OUT + / OUT -	speaker only (never GND)	SOURCE

6 Single flex-sensor circuit

DIAGRAM 2: ONE FLEX SENSOR (voltage divider)



Only the ADC wire leaves the node. Nothing else is connected to the node except the sensor tab and the resistor.
Example numbers in the source PDF (about 3194 straight) are EXAMPLES. Use your own measurements.

Diagram 2. One divider: 3.3 V → flex sensor → ADC node → 10 kΩ → GND. $V_{out} = 3.3 \text{ V} \times 10\text{k} / (R_{flex} + 10\text{k})$.

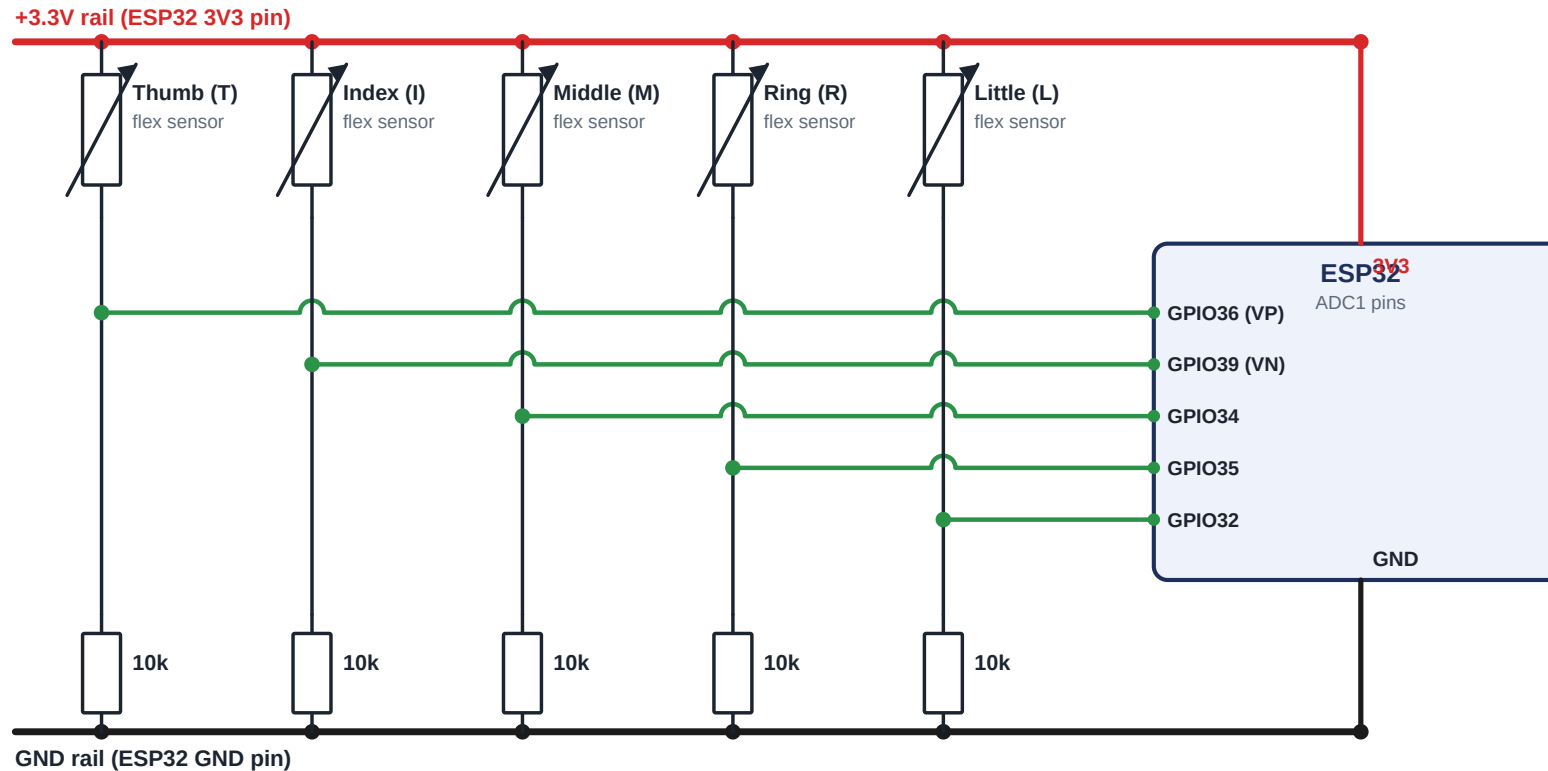
6 Single flex sensor: what to expect

- A straight sensor of about 25 k Ω with a 10 k Ω resistor gives $V_{out} \approx 3.3 \times 10 / 35 = \mathbf{0.94\ V}$, about **1170 counts** on the 12-bit ADC (0–4095). Individual sensors differ a lot, so treat this as a sanity check, not a threshold.
- Bending raises the sensor resistance, so V_{out} and the ADC reading **fall**. Example: 60 k Ω gives 0.47 V, about 580 counts.
- The source PDF's example numbers (about 3194 straight, 1351 bent) are examples from a different assumed sensor. Use **your own** readings.
- The ADC node has exactly three things on it: the sensor tab 2, the top of the 10 k Ω resistor, and the green ADC wire to the ESP32. Nothing else.
- **Sanity test on the bench:** replace the flex sensor with a second 10 k Ω resistor. The ADC should read about half scale (roughly 2000). Replace it with a plain jumper wire: about 4095. Remove the resistor: about 0.

Test sketch (Serial Monitor 115200): `analogReadResolution(12); analogSetAttenuation(ADC_11db); Serial.println(analogRead(36));` — use GPIO36 (VP) for the thumb. The full five-channel sketch is `flex_five_sensors_test.ino`.

7 Five-flex-sensor circuit

DIAGRAM 3: FIVE FLEX SENSORS - five independent voltage dividers



Each branch: 3.3V > flex sensor > NODE > 10k > GND. The green ADC wire leaves the node only. Green arcs = wires crossing WITHOUT touching. The five nodes are never joined.

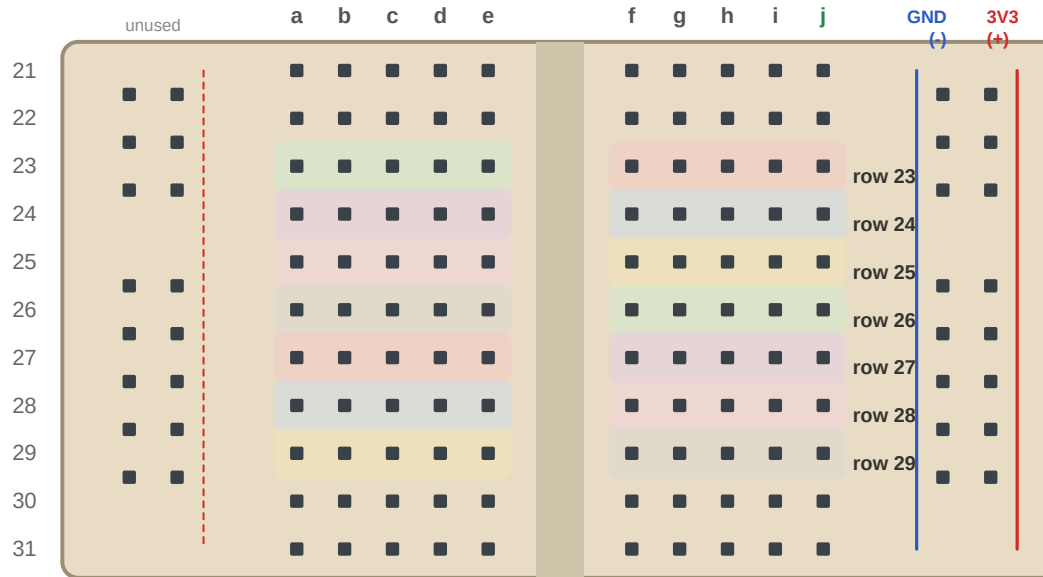
PROTOTYPE WORKAROUND

Breadboard prototype only: the 3V3 pin is under the ESP32 body, so the rail is fed from D33 (GPIO33 = HIGH). See Section 6. On the glove perfboard use the real 3V3 pin.

Diagram 3. Five independent dividers. They share only the 3.3 V and GND rails; the five ADC nodes are never joined.

8 Breadboard wiring: which holes are connected

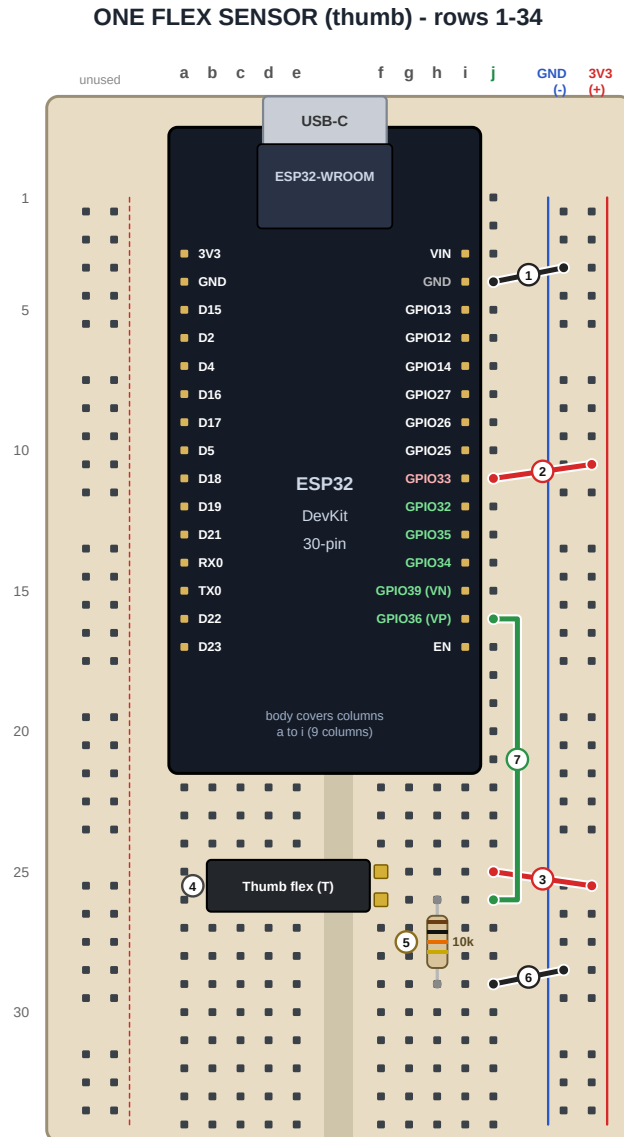
WHICH HOLES ARE CONNECTED (top view, ESP32 end at the top)



Breadboard geometry (the layout used in Diagrams 13–16)

- The board is used **portrait**, with the ESP32's USB-C connector at the **top**. Row 1 is the end nearest the USB-C connector. The ESP32 is never rotated.
- Each numbered row has two groups of five holes: **a b c d e** (left) and **f g h i j** (right). **The five holes of one group in one row are joined inside the board.** The left group and the right group are **not** joined to each other (the centre trench).
- Different rows are **never** joined. Row 25 and row 26 are different strips even though they are neighbours.
- The two long **power rails** along the edge run the full length, top to bottom. In this manual the right-hand rails are used: inner rail = **GND (blue line)**, outer rail = **3.3 V (red line)**. Some boards split each rail in the middle. **Test with a multimeter (continuity)** and bridge the gap if it exists.
- If your board is turned sideways (landscape), the same rule applies: the 5-hole groups become vertical columns and the rails run horizontally.
- The 30-pin ESP32 sits with its pin rows in **column a** and **column i**: it covers **9 of the 10** middle columns. **Column j is the one free column**: every ESP32-pin jumper on the right side goes into column j (the same strip as pin column i). The left-side pins (including 3V3) are unreachable on one board, so **D33 (GPIO33, driven HIGH by the sketch) supplies the 3.3 V** for the flex dividers. This is fine: five dividers draw about 0.5 mA in total.

8 One-flex breadboard diagram



Build ONE finger (the thumb) in this order

1. Unplug USB. Seat the ESP32 (USB-C up) so its pin rows are in **columns a and i**.
2. **Jumper 1 (black)**: ESP32 GND pin row (row 4), hole **j4** → GND rail.
3. **Jumper 2 (red)**: ESP32 D33 pin row (row 11), hole **j11** → 3.3 V rail. (The sketch drives D33 HIGH.)
4. **Jumper 3 (red)**: hole **j25** → 3.3 V rail. Row 25 now carries 3.3 V.
5. Push **flex tab 1** into **f25** and **tab 2** into **f26** (two different rows).
6. 10 kΩ resistor: one lead in **h26**, the other in **h29**.
7. **Jumper 6 (black)**: **j29** → GND rail.
8. **Jumper 7 (green)**: **j26** → **j16** (ESP32 VP = GPIO36 pin row).
9. Before USB: continuity-check, then measure 3.3 V between the rails with USB plugged in (Section 9).

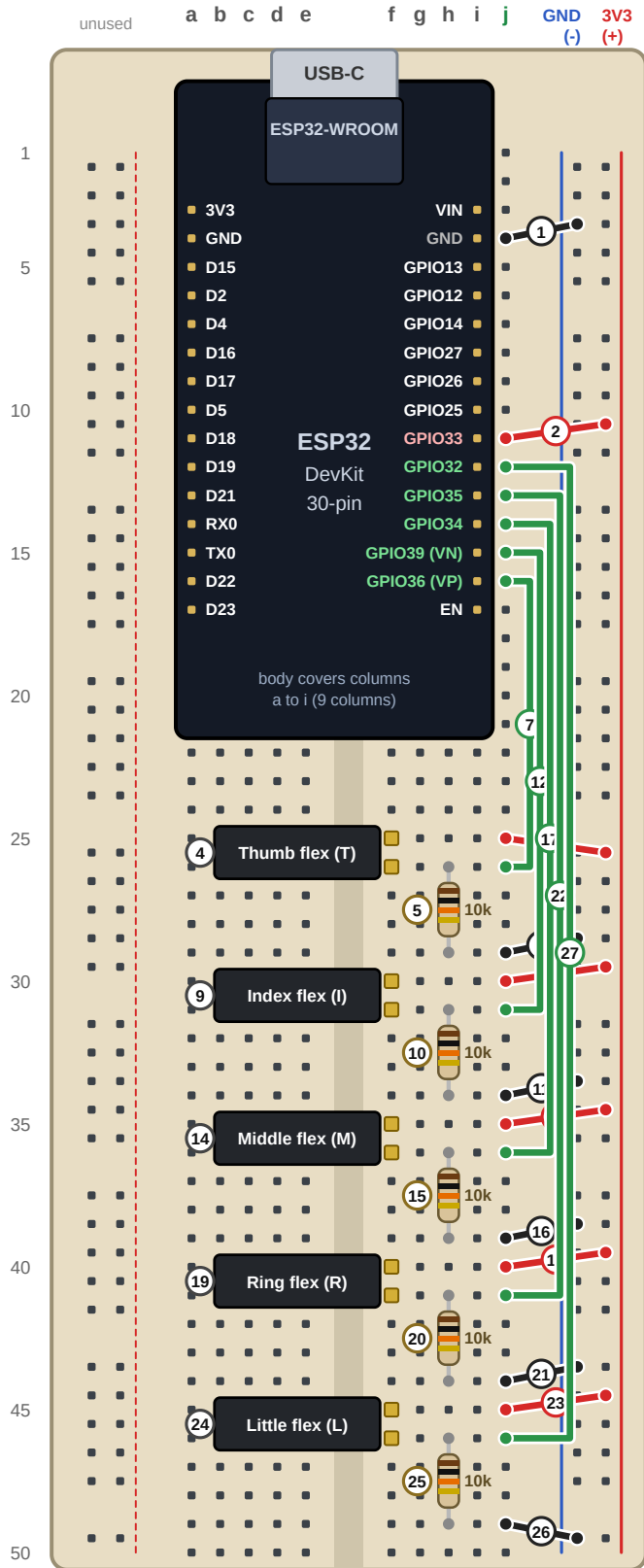
8 One-flex wiring table (matches Diagram 13 numbers)

#	Part	From	To	Purpose
1	Black jumper wire	j4 (column j, row 4)	GND rail (inner, blue line) beside row 3.5	ESP32 GND pin to GND rail
2	Red jumper wire	j11 (column j, row 11)	3.3 V rail (outer, red line) beside row 10.5	ESP32 D33 (HIGH = 3.3 V) to 3.3 V rail
3	Red jumper wire	j25 (column j, row 25)	3.3 V rail (outer, red line) beside row 25.5	Thumb: row 25 to 3.3 V rail
4	Flex sensor	tab 1 in f25 (column f, row 25)	tab 2 in f26 (column f, row 26)	Thumb flex sensor
5	10 kΩ resistor (brown- black-orange)	lead 1 in h26 (column h, row 26)	lead 2 in h29 (column h, row 29)	10 k ohm
6	Black jumper wire	j29 (column j, row 29)	GND rail (inner, blue line) beside row 28.5	Thumb: row 29 to GND rail
7	Green jumper wire	j26 (column j, row 26)	j16 (column j, row 16)	Thumb: node to ESP32 GPIO36 (VP)

The table is generated from the same data as the diagram, so a wire number here is the same wire number in the picture. Rows are counted from the ESP32 (USB-C) end.

8 Five-flex breadboard diagram

FIVE FLEX SENSORS - rows 1-50



Five fingers

Every finger repeats the one-finger block, five rows lower each time:

Finger	3.3 V row	Node row	GND row	ADC jumper → pin row
Thumb	25	26	29	j26 → j16 (GPIO36 (VP))
Index	30	31	34	j31 → j15 (GPIO39 (VN))
Middle	35	36	39	j36 → j14 (GPIO34 (D34))
Ring	40	41	44	j41 → j13 (GPIO35 (D35))
Little	45	46	49	j46 → j12 (GPIO32 (D32))

- Add fingers **one at a time**; after each one check that only that finger's value changes when you bend it.
- The five green ADC jumpers run in separate lanes and are never plugged into the same row. Wires cross *above* each other; crossing is fine as long as the ends go to their own holes.
- You need a full-length board: the last block ends at row 49.

8 Five-flex wiring table (matches Diagram 14 numbers)

#	Part	From	To	Purpose
1	Black jumper wire	j4 (column j, row 4)	GND rail (inner, blue line) beside row 3.5	ESP32 GND pin to GND rail
2	Red jumper wire	j11 (column j, row 11)	3.3 V rail (outer, red line) beside row 10.5	ESP32 D33 (HIGH = 3.3 V) to 3.3 V rail
3	Red jumper wire	j25 (column j, row 25)	3.3 V rail (outer, red line) beside row 25.5	Thumb: row 25 to 3.3 V rail
4	Flex sensor	tab 1 in f25 (column f, row 25)	tab 2 in f26 (column f, row 26)	Thumb flex sensor
5	10 kΩ resistor (brown-black-orange)	lead 1 in h26 (column h, row 26)	lead 2 in h29 (column h, row 29)	10 k ohm
6	Black jumper wire	j29 (column j, row 29)	GND rail (inner, blue line) beside row 28.5	Thumb: row 29 to GND rail
7	Green jumper wire	j26 (column j, row 26)	j16 (column j, row 16)	Thumb: node to ESP32 GPIO36 (VP)
8	Red jumper wire	j30 (column j, row 30)	3.3 V rail (outer, red line) beside row 29.5	Index: row 30 to 3.3 V rail
9	Flex sensor	tab 1 in f30 (column f, row 30)	tab 2 in f31 (column f, row 31)	Index flex sensor
10	10 kΩ resistor (brown-black-orange)	lead 1 in h31 (column h, row 31)	lead 2 in h34 (column h, row 34)	10 k ohm
11	Black jumper wire	j34 (column j, row 34)	GND rail (inner, blue line) beside row 33.5	Index: row 34 to GND rail
12	Green jumper wire	j31 (column j, row 31)	j15 (column j, row 15)	Index: node to ESP32 GPIO39 (VN)
13	Red jumper wire	j35 (column j, row 35)	3.3 V rail (outer, red line) beside row 34.5	Middle: row 35 to 3.3 V rail
14	Flex sensor	tab 1 in f35 (column f, row 35)	tab 2 in f36 (column f, row 36)	Middle flex sensor
15	10 kΩ resistor (brown-black-orange)	lead 1 in h36 (column h, row 36)	lead 2 in h39 (column h, row 39)	10 k ohm
16	Black jumper wire	j39 (column j, row 39)	GND rail (inner, blue line) beside row 38.5	Middle: row 39 to GND rail
17	Green jumper wire	j36 (column j, row 36)	j14 (column j, row 14)	Middle: node to ESP32 GPIO34 (D34)
18	Red jumper wire	j40 (column j, row 40)	3.3 V rail (outer, red line) beside row 39.5	Ring: row 40 to 3.3 V rail
19	Flex sensor	tab 1 in f40 (column f, row 40)	tab 2 in f41 (column f, row 41)	Ring flex sensor

#	Part	From	To	Purpose
20	10 kΩ resistor (brown-black-orange)	lead 1 in h41 (column h, row 41)	lead 2 in h44 (column h, row 44)	10 k ohm
21	Black jumper wire	j44 (column j, row 44)	GND rail (inner, blue line) beside row 43.5	Ring: row 44 to GND rail
22	Green jumper wire	j41 (column j, row 41)	j13 (column j, row 13)	Ring: node to ESP32 GPIO35 (D35)
23	Red jumper wire	j45 (column j, row 45)	3.3 V rail (outer, red line) beside row 44.5	Little: row 45 to 3.3 V rail
24	Flex sensor	tab 1 in f45 (column f, row 45)	tab 2 in f46 (column f, row 46)	Little flex sensor
25	10 kΩ resistor (brown-black-orange)	lead 1 in h46 (column h, row 46)	lead 2 in h49 (column h, row 49)	10 k ohm
26	Black jumper wire	j49 (column j, row 49)	GND rail (inner, blue line) beside row 49.5	Little: row 49 to GND rail
27	Green jumper wire	j46 (column j, row 46)	j12 (column j, row 12)	Little: node to ESP32 GPIO32 (D32)

8 Correct vs incorrect breadboard wiring

CORRECT vs INCORRECT breadboard wiring (side by side) - one flex sensor, rows 21-31

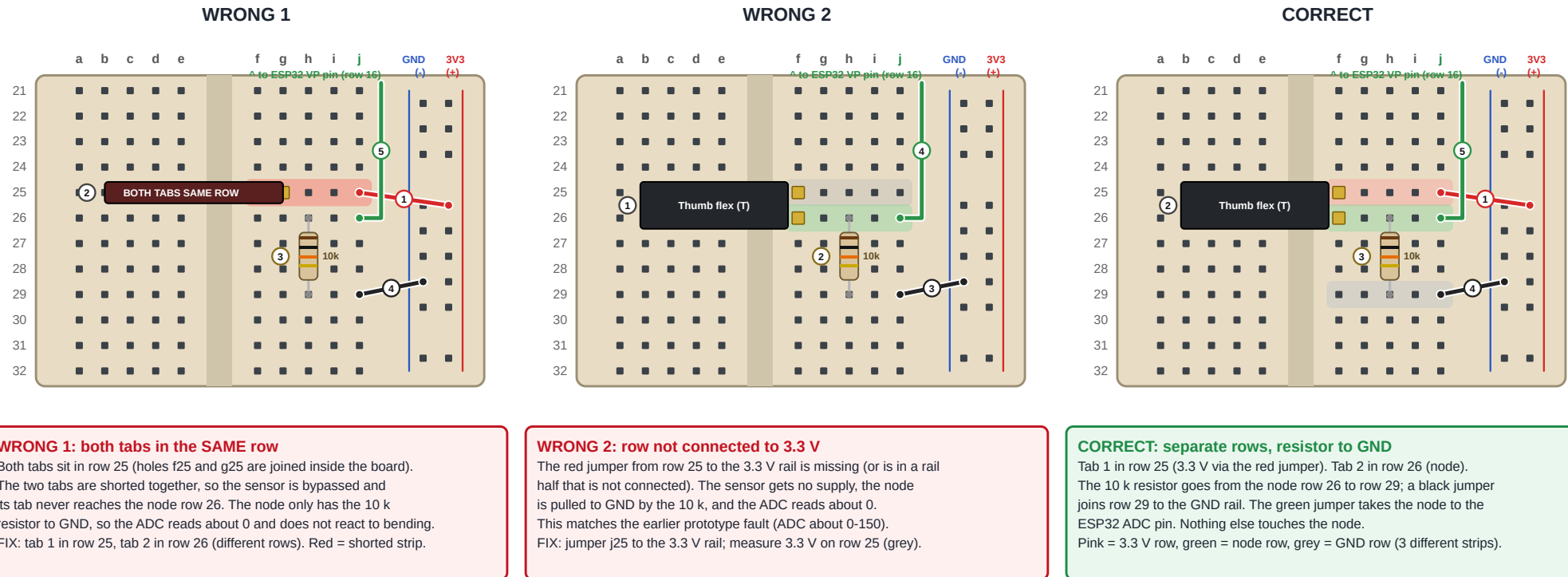


Diagram 15. Left and centre: the two faults that give an ADC reading near 0. Right: the correct wiring.

Why "same row" fails: holes f25 and g25 are one strip. A sensor with both tabs in it is shorted, and the row that should be the node (row 26) then only sees the 10 kΩ resistor to GND. **Rule:** the two tabs of one sensor must always be in two different rows.

9 Previous wiring failure analysis

Symptom seen	What it tells us
ADC readings about 0–150, random, no steady value	The ADC pins are not held at a real voltage: nearly floating, or held to GND only by the 10 kΩ resistor with no supply above it.
Bending or touching one sensor moved all channels	Floating ADC inputs pick up their neighbours and your hand. Not evidence of a series/parallel design error.
A 10 kΩ resistor in place of a sensor still read about 0	The resistor and sensor were not the fault: the 3.3 V never reached that row.
Averaging + settle delays made no difference	Not an ADC-multiplexer crosstalk problem. The electrical supply is missing.
Switching D33 off and on changed nothing lasting (only a short blip at switch-on)	D33 was not delivering a lasting voltage to the sensor rows.

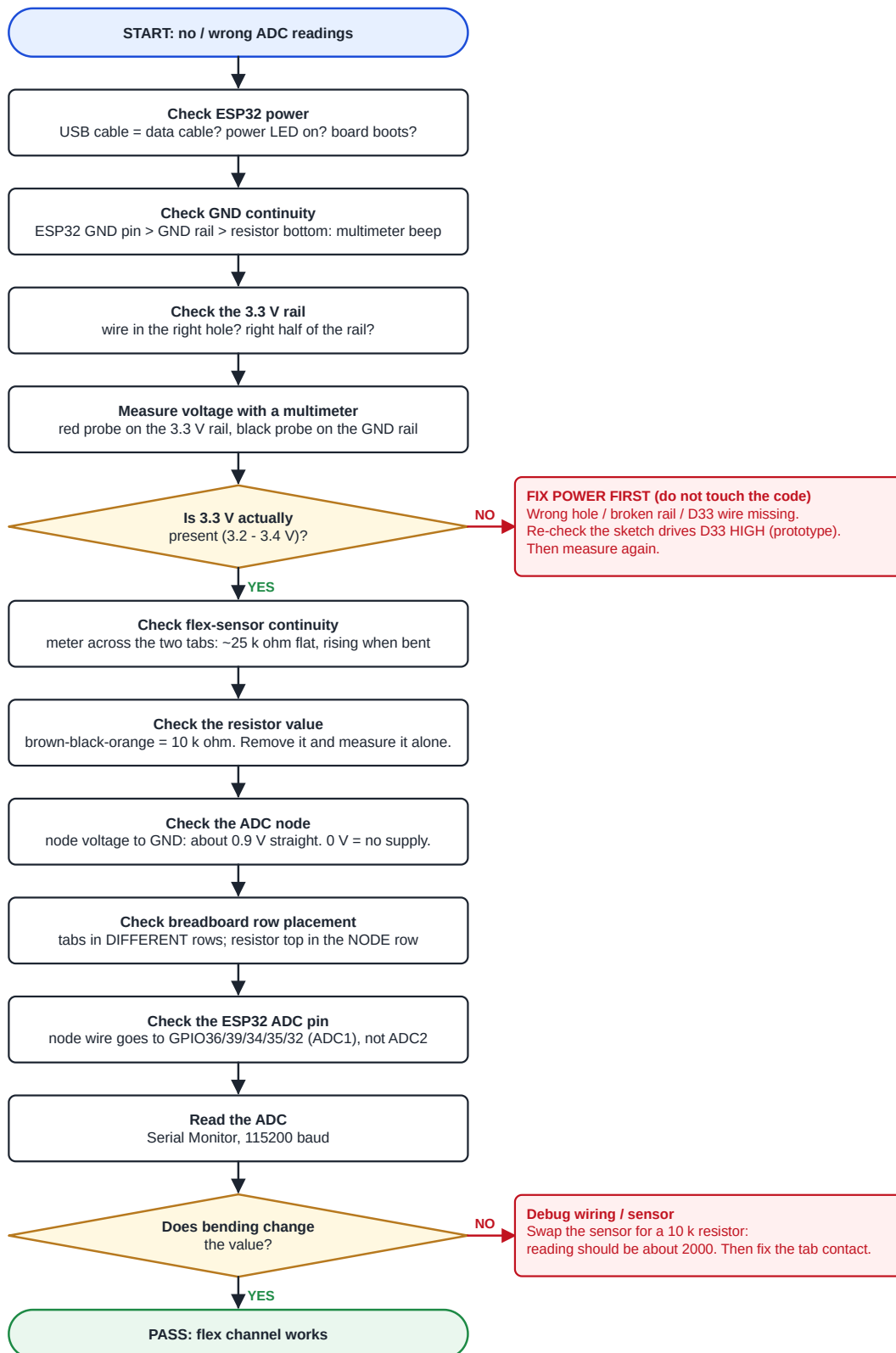
Conclusion (from the progress report): the likely problem is that **3.3 V was not actually reaching the sensor rows**. Causes to rule out, in order of likelihood: (1) the supply jumper is in the wrong hole or a row that is not the sensors' supply row; (2) the 3.3 V rail has a break in the middle; (3) loose flex-sensor tabs; (4) the D33 supply wire is not seated (check the D33 jumper and that the sketch drives D33 HIGH).

Rule for this project: do not change code while the electrical supply has not been verified with a multimeter. Follow the flowchart on the next page, then the multimeter table.

9 Flex-sensor wiring troubleshooting flowchart

FLEX-SENSOR WIRING TROUBLESHOOTING FLOWCHART

Measure the electrical supply BEFORE changing any code.



9 Multimeter debugging guide

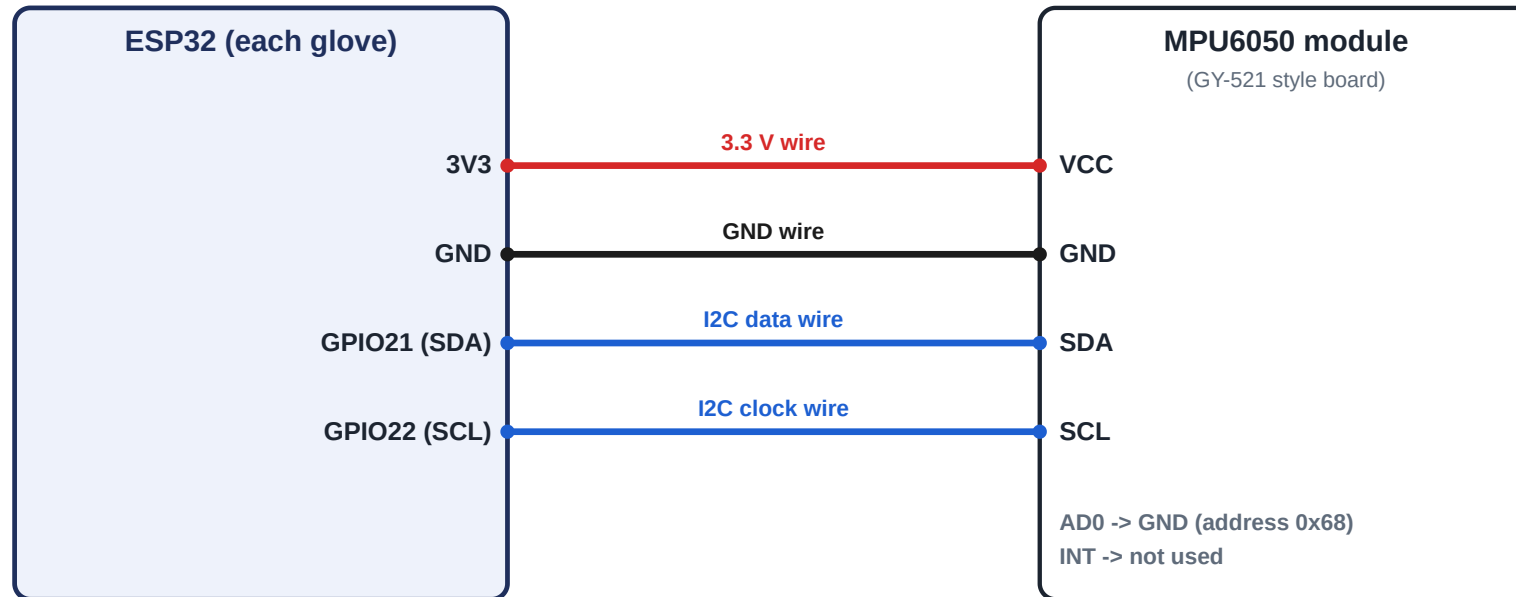
Black probe = COM jack, **red probe** = V Ω jack. Volts tests: USB plugged in and the sketch running (DC volts range, 20 V). Resistance/continuity tests: **USB unplugged**. Never measure resistance on a powered circuit.

#	TEST	PROBE PLACEMENT	EXPECTED RESULT	IF FAILED	ACTION
1	ESP32 3.3 V -> GND	USB plugged in. Red probe on the ESP32 3V3 pin (left row, top), black probe on the ESP32 GND pin.	3.2 - 3.4 V	Reading 0 V or below 3.0 V	Try another USB cable/port; look for a short (hot part); unplug everything except the ESP32 and retest.
2	Sensor supply -> GND (prototype: D33 net)	Red probe on the 3.3 V rail (or the row holding flex tab 1), black on the GND rail. The D33 sketch must be running.	3.2 - 3.4 V	About 0 V	D33 jumper in the wrong hole / wrong rail half. Move it; make sure the sketch sets D33 HIGH.
3	ADC node -> GND	Red probe on the node row (flex tab 2 / resistor top), black on the GND rail. Finger straight.	About 0.7 - 1.0 V with a 25 k sensor (0.9 V typical)	0 V = no supply; 3.3 V = resistor open / no GND path	Follow the supply path; re-seat the resistor to GND.
4	Resistor continuity / value	Power OFF, remove the resistor. Ohms range, probes on the two leads.	9.5 k - 10.5 k ohm (brown-black-orange)	Open (OL) or a wrong value	Replace the resistor; check the colour bands.
5	Flex sensor continuity	Power OFF. Ohms range, probes on the two tabs. Flat, then bent.	About 25 k ohm flat; rising as it bends (sensor dependent)	OL (open) or almost 0 (short)	Loose tabs are common: re-seat, or solder wires to the tabs. Replace the sensor if open.
6	Ground continuity	Power OFF. Continuity (beep) mode: ESP32 GND pin to the GND rail, then to the resistor bottom.	Beep (< 1 ohm) at each point	No beep	GND jumper in wrong hole / wrong rail. Rail may be split in the middle: bridge it.
7	Power-rail continuity	Power OFF. Continuity mode along the 3.3 V rail from the first hole to the last hole used. Do the same for GND.	Beep along the whole rail	Beep only for the first half	Your rail is split in the middle: add a short jumper across the gap.
8	ADC pin continuity	Power OFF. Continuity mode: node row to the ESP32 pin row (e.g. GPIO36/VP), using the strip that the jumper connects.	Beep	No beep	ADC jumper not seated or in the wrong pin row. Compare to the wiring table.

DIAGRAM 4: MPU6050 + ESP32 (I2C) - same wiring on both gloves

WIRE COLOUR CODE (every wire is also text-labelled)

— 3.3 V — GND — I2C

**SOURCE-DOCUMENT CONNECTIONS**

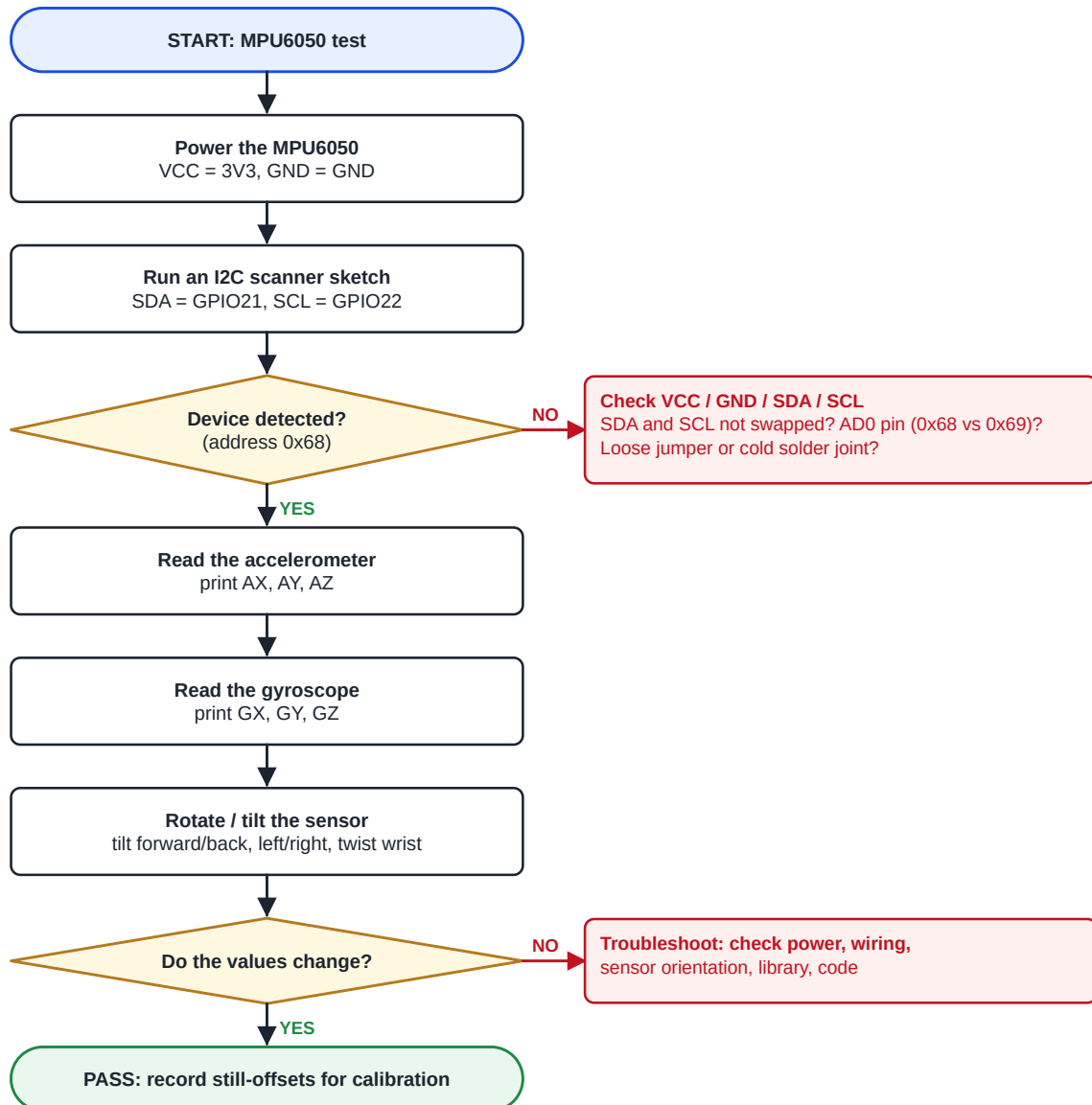
Pins in the source PDF (Figure 8): ESP32 GPIO21 = SDA, GPIO22 = SCL.
I2C address 0x68 (AD0 low) or 0x69 (AD0 high).
Keep the four wires short and twisted.
The order of pins on YOUR module can differ.

VERIFY ON ACTUAL MODULE

Check the silkscreen for VCC / GND / SCL / SDA on the actual board.
Confirm the module accepts 3.3 V on VCC (most GY-521 boards do).
On-board I2C pull-up resistors: VERIFY they exist on your module.
Mount flat on the back of the hand; axis arrows are printed on the board.

Diagram 4. Same wiring on both gloves.

MPU6050 TEST FLOWCHART



I2C scanner (expect 0x68)

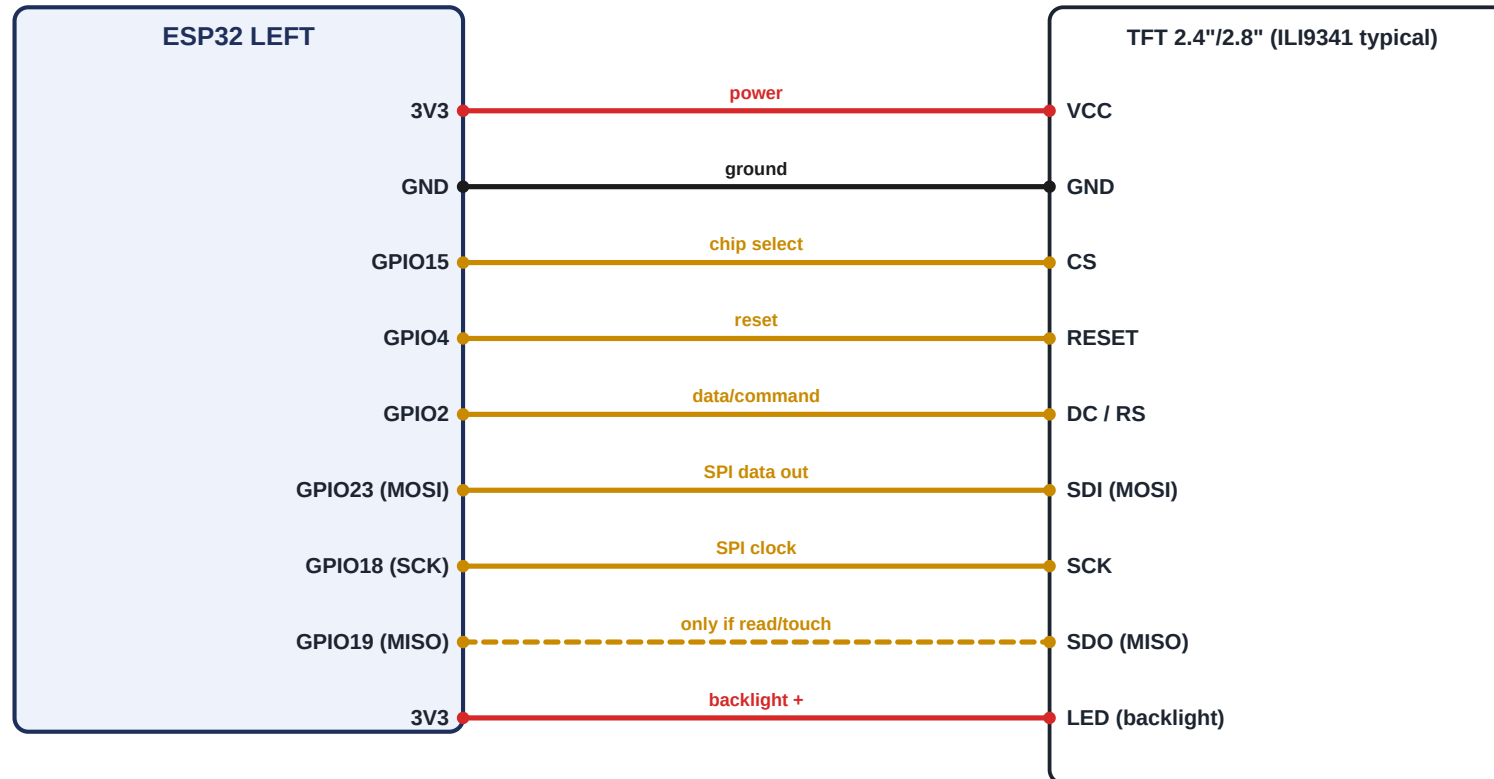
```
#include <Wire.h>
void setup() { Serial.begin(115200); Wire.begin(21, 22); } // SDA=21, SCL=22
void loop() {
  int found = 0;
  for (byte a = 1; a < 127; a++) { Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) { Serial.printf("Found 0x%02X\n", a); found++; } }
  if (!found) Serial.println("No I2C devices found");
  delay(2000);
}
```

Movement test (source PDF, Fig. 9)

Hold still, palm down	AZ $\approx$ +1 g, GX/GY/GZ $\approx$ 0
Tilt forward / back	AY changes sign (pitch)
Tilt left / right	AX changes sign (roll)
Twist wrist	GZ spikes while rotating
Still for 5 s	record offsets and subtract them

Axis arrows are printed on the module: check them. Repeat the wiring, address check and calibration on the second glove.

DIAGRAM 5: TFT (ILI9341, SPI) + ESP32 LEFT

**SOURCE-DOCUMENT CONNECTIONS**

Source PDF (Figure 10): pins match the common TFT_eSPI ESP32 setup.
Match them in TFT_eSPI User_Setup.h.
MISO is only needed for touch / read-back (dashed).

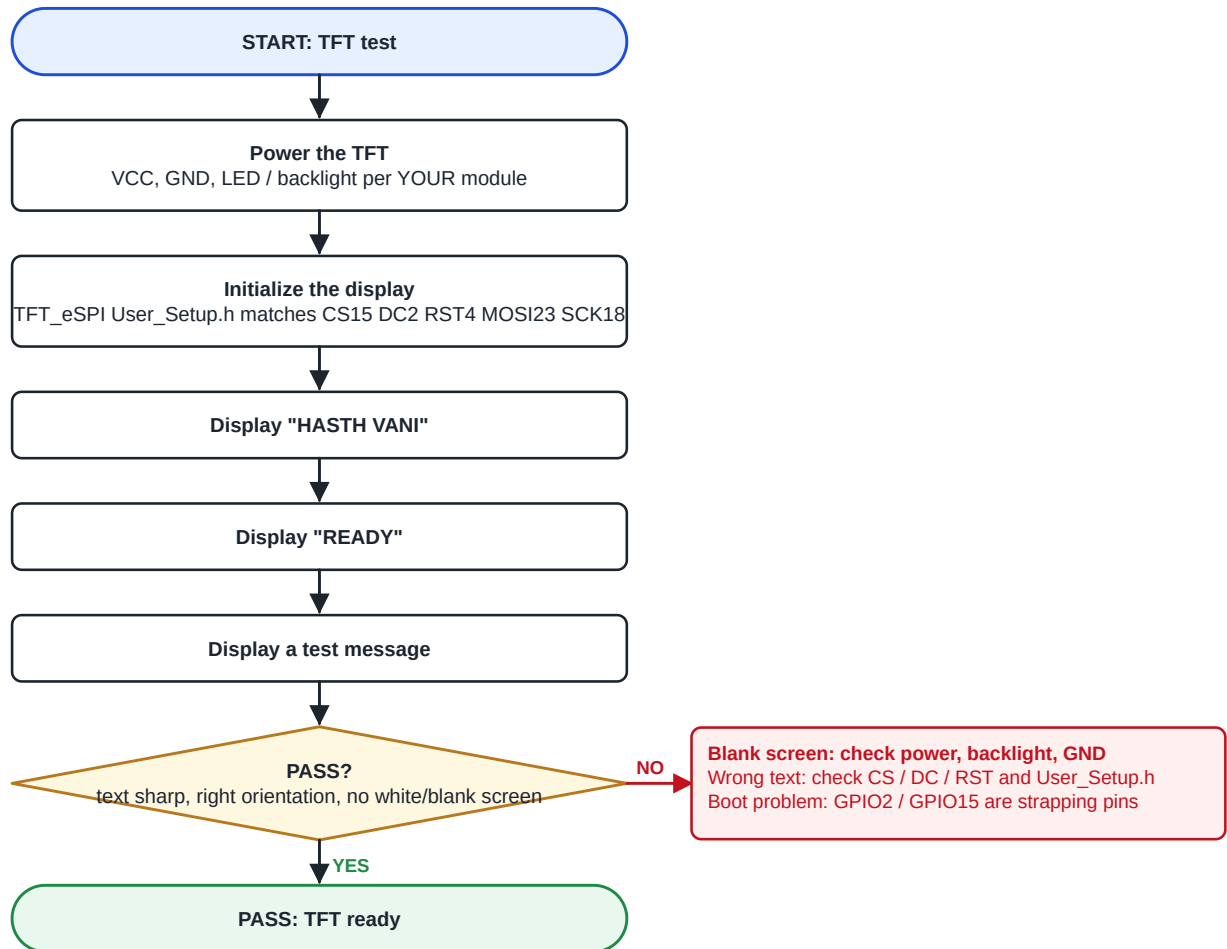
VERIFY ON ACTUAL MODULE

VERIFY THE TFT MODULE PINOUT before permanent wiring.
LED / backlight: source says 3.3 V (or via 100 ohm / transistor). Check YOUR module.
GPIO2 and GPIO15 are ESP32 boot-strapping pins: if flashing fails, unplug them once.

Diagram 5. ESP32 LEFT to a typical ILI9341 SPI TFT.

11 TFT test

TFT TEST FLOWCHART



Display test

```
// TFT_eSPI User_Setup.h (typical ILI9341; VERIFY your display driver)
#define ILI9341_DRIVER
#define TFT_MISO 19
#define TFT_MOSI 23
#define TFT_SCLK 18
#define TFT_CS 15
#define TFT_DC 2
#define TFT_RST 4

#include <TFT_eSPI.h>
TFT_eSPI tft;
void setup() {
  tft.init(); tft.setRotation(1); tft.fillScreen(TFT_BLACK);
  tft.setTextColor(TFT_WHITE); tft.setTextSize(3);
  tft.drawString("HASTH VANI", 10, 10); tft.drawString("READY", 10, 60);
  tft.setTextSize(2); tft.drawString("TEST MESSAGE 123", 10, 110);
}
void loop() {}
```

VERIFY THE TFT MODULE PINOUT before permanent wiring. Pin names, the backlight (LED) supply, and the driver chip differ between modules. GPIO2 and GPIO15 are ESP32 boot-strapping pins: if the board will not start or upload while the TFT is connected, unplug DC (GPIO2) or CS (GPIO15) for the upload only.

DIAGRAM 6: MAX98357A + ESP32 RIGHT + 3 W SPEAKER

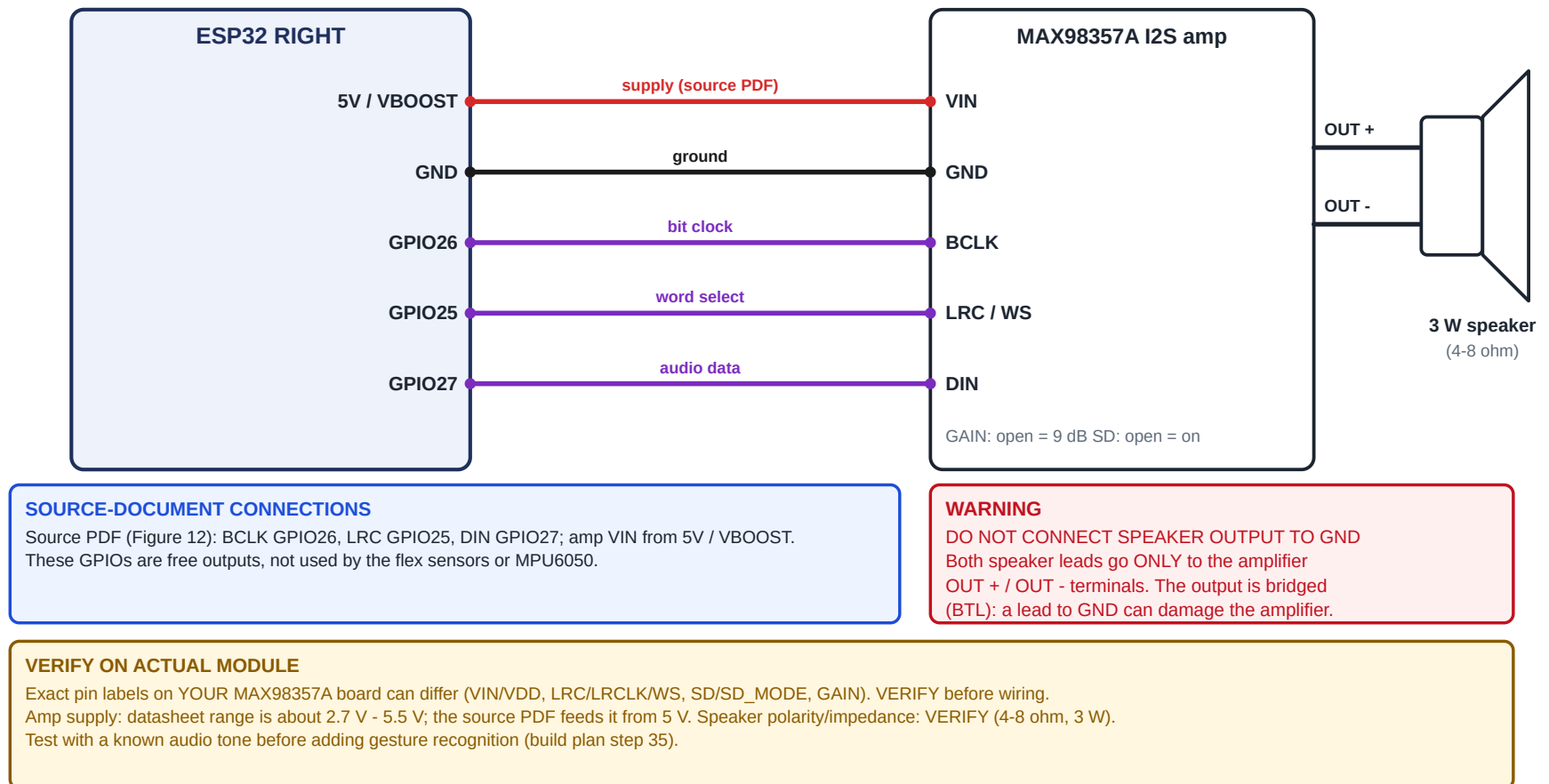


Diagram 6. I2S amplifier and 3 W speaker on the RIGHT ESP32.

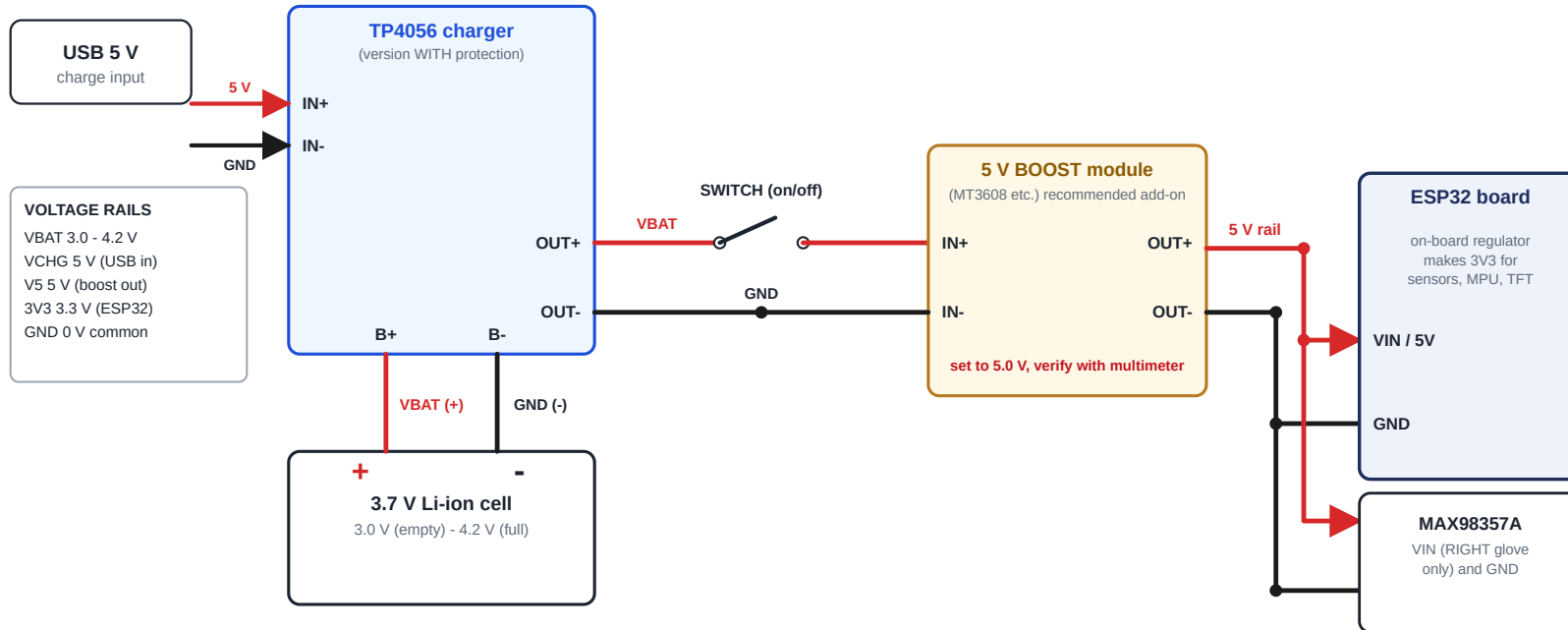
DO NOT CONNECT SPEAKER OUTPUT TO GND. The amplifier output is bridged (BTL). Connect the two speaker leads only to the amplifier's marked speaker output terminals. Shorting either lead to GND can damage the amplifier.

- I2S signals: **BCLK = GPIO26, LRC/WS = GPIO25, DIN = GPIO27**. GAIN and SD left open (9 dB, amplifier on) as in the source PDF.
- Amplifier supply: the source PDF feeds it from **5 V** (VBOOST). The MAX98357A datasheet supply range is about 2.7–5.5 V. **VERIFY** on your board and use one supply consistently.
- Speaker: 3 W, 4–8 Ω . Check polarity and never short the two outputs together.
- Test with a **known audio tone** before adding gesture recognition (build plan step 35). Check volume, clarity and that the amplifier does not get hot.
- Audio code depends on the ESP32 core version (I2S API changed between core 2.x and 3.x): use the example that matches your installed core. **VERIFY**.

13 Power system

DIAGRAM 9: POWER AND CHARGING (one set per glove)

LEFT and RIGHT gloves each have their own battery + TP4056 + switch + boost. Nothing is shared between gloves.



COMMON GROUND: every module GND joins TP4056 OUT- / battery -

WIRE COLOUR CODE (every wire is also text-labelled)

— positive / supply — GND

SAFETY

1. Set the boost to 5.0 V with a multimeter BEFORE connecting any load.
2. Check battery and TP4056 polarity (+ to +, - to -) before connecting.
3. Never feed 5 V into a 3V3 pin. Never power a 3.3 V module from VBAT or 5 V.
4. Do not run USB and the boost into the ESP32 at the same time (VERIFY back-feed).

ITEM TO VERIFY

VERIFY EXACT MODULE POWER REQUIREMENTS BEFORE CONNECTION.
A single 3.7 V cell is below what many ESP32 boards' VIN regulators need, so the source PDF recommends a 5 V boost (its Figure 15). The boost is NOT in the inventory: obtain one and confirm your ESP32 board's VIN range.

Diagram 9. One power set per glove. Battery → TP4056 → switch → 5 V boost → ESP32 VIN; ESP32 3V3 feeds the sensors.

13 Power system notes

Rail	Voltage	Where from	Feeds
VCHG	5 V (USB)	USB charger	TP4056 IN+ / IN-
VBAT	3.0 – 4.2 V	Li-ion cell (via TP4056 OUT+)	switch, then the boost IN+
V5	5.0 V (set and measured)	5 V boost OUT+	ESP32 VIN / 5V pin, MAX98357A VIN (RIGHT only)
3V3	3.3 V	ESP32 on-board regulator	flex dividers, MPU6050, TFT
GND	0 V	TP4056 OUT- / battery -	every module

Safety. (1) Set the boost to 5.0 V with a multimeter before connecting any load. (2) Check battery and TP4056 polarity before connecting. (3) Never feed 5 V or battery voltage into a 3V3 pin or to a 3.3 V-only module. (4) Do not power the ESP32 from USB and from the boost at once without confirming your board prevents back-feeding. (5) Test the TP4056 and battery separately first (build-plan step 52). (6) Insulate every joint.

VERIFY EXACT MODULE POWER REQUIREMENTS BEFORE CONNECTION. TP4056 module variant (with or without protection), boost module type and output setting, ESP32 board VIN input range, and TFT/MPU6050/MAX98357A supply ranges must be confirmed from the actual parts.

14 Left glove complete circuit

DIAGRAM 7: LEFT GLOVE COMPLETE CIRCUIT (display glove)

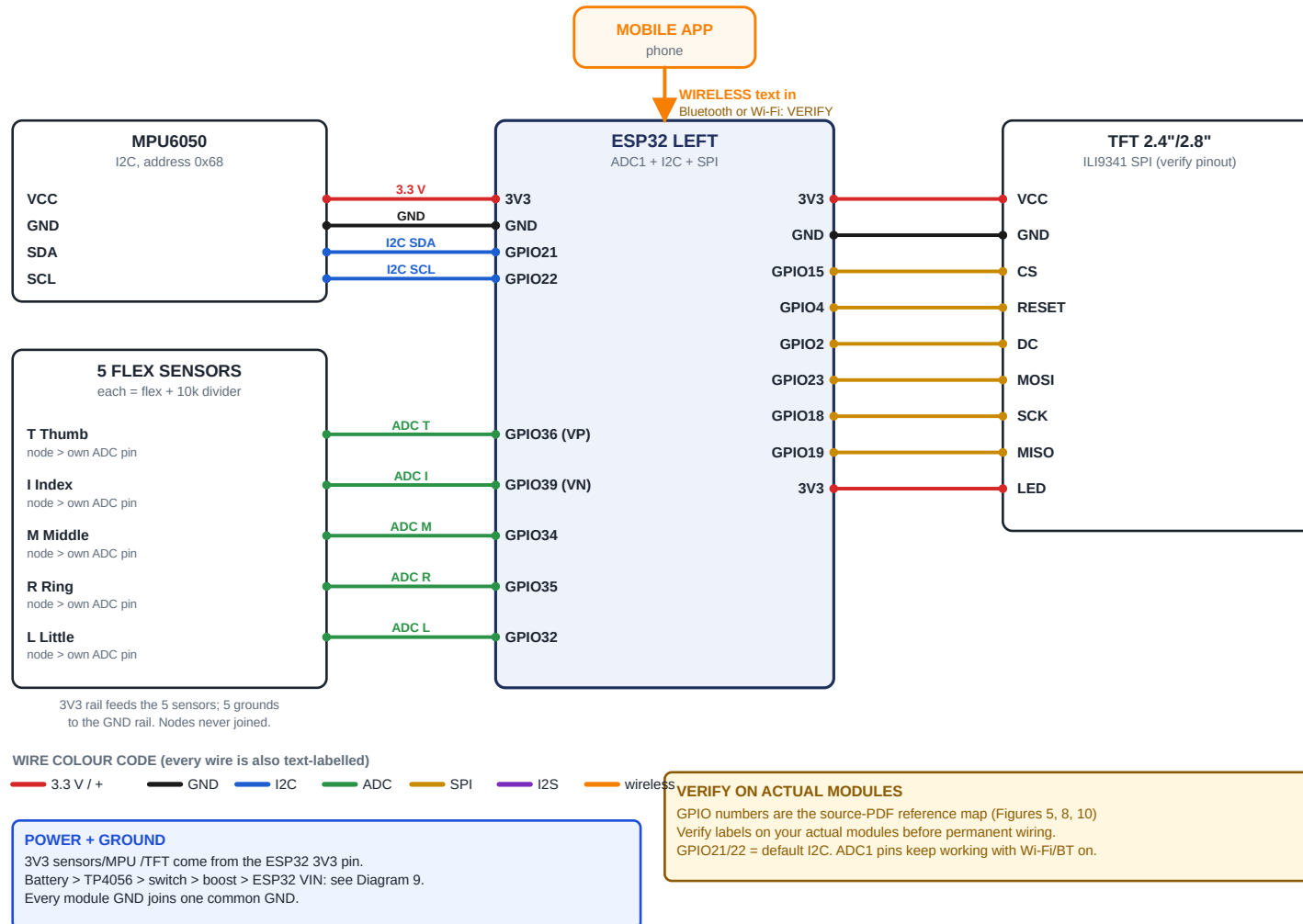


Diagram 7. 5 flex + MPU6050 + TFT + wireless on ESP32 LEFT.

15 Right glove complete circuit

DIAGRAM 8: RIGHT GLOVE COMPLETE CIRCUIT (speaker glove)

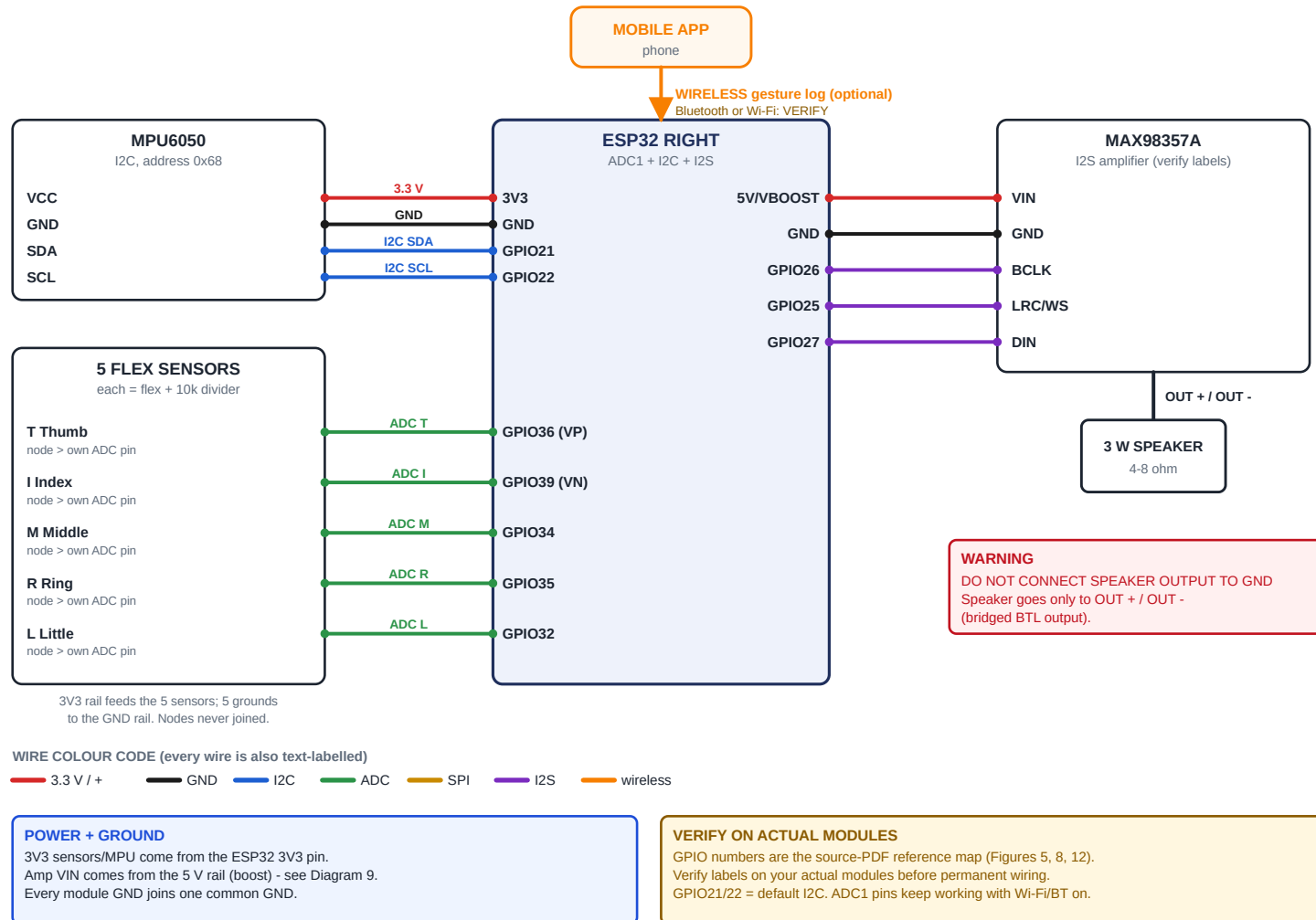


Diagram 8. 5 flex + MPU6050 + MAX98357A + speaker on ESP32 RIGHT.

16 Complete two-glove system

DIAGRAM 10: COMPLETE TWO-GLOVE SYSTEM

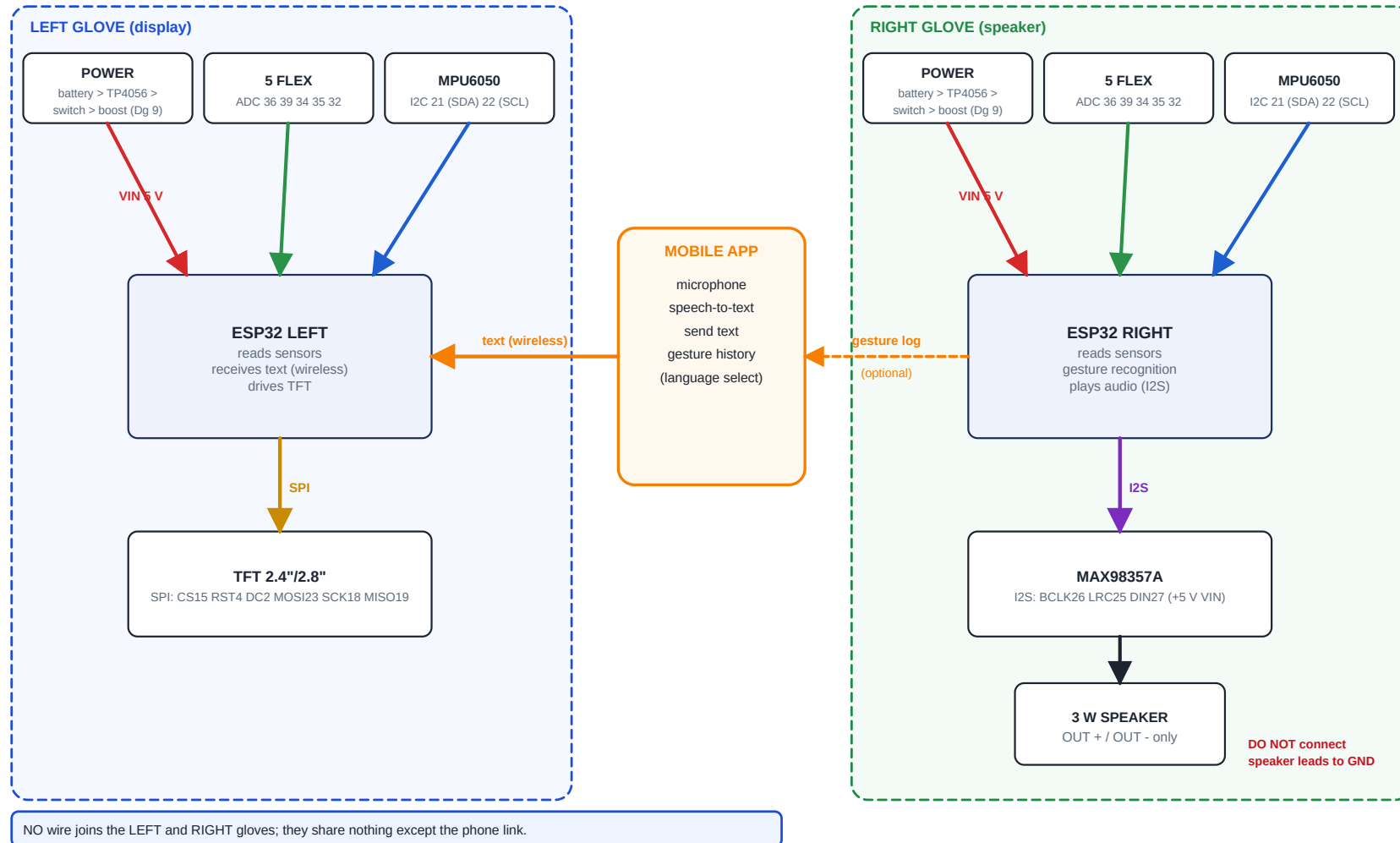


Diagram 10. Two independent glove systems, each with its own power; linked only through the phone.

17 Mobile app communication

DIAGRAM 11: MOBILE APP COMMUNICATION FLOW

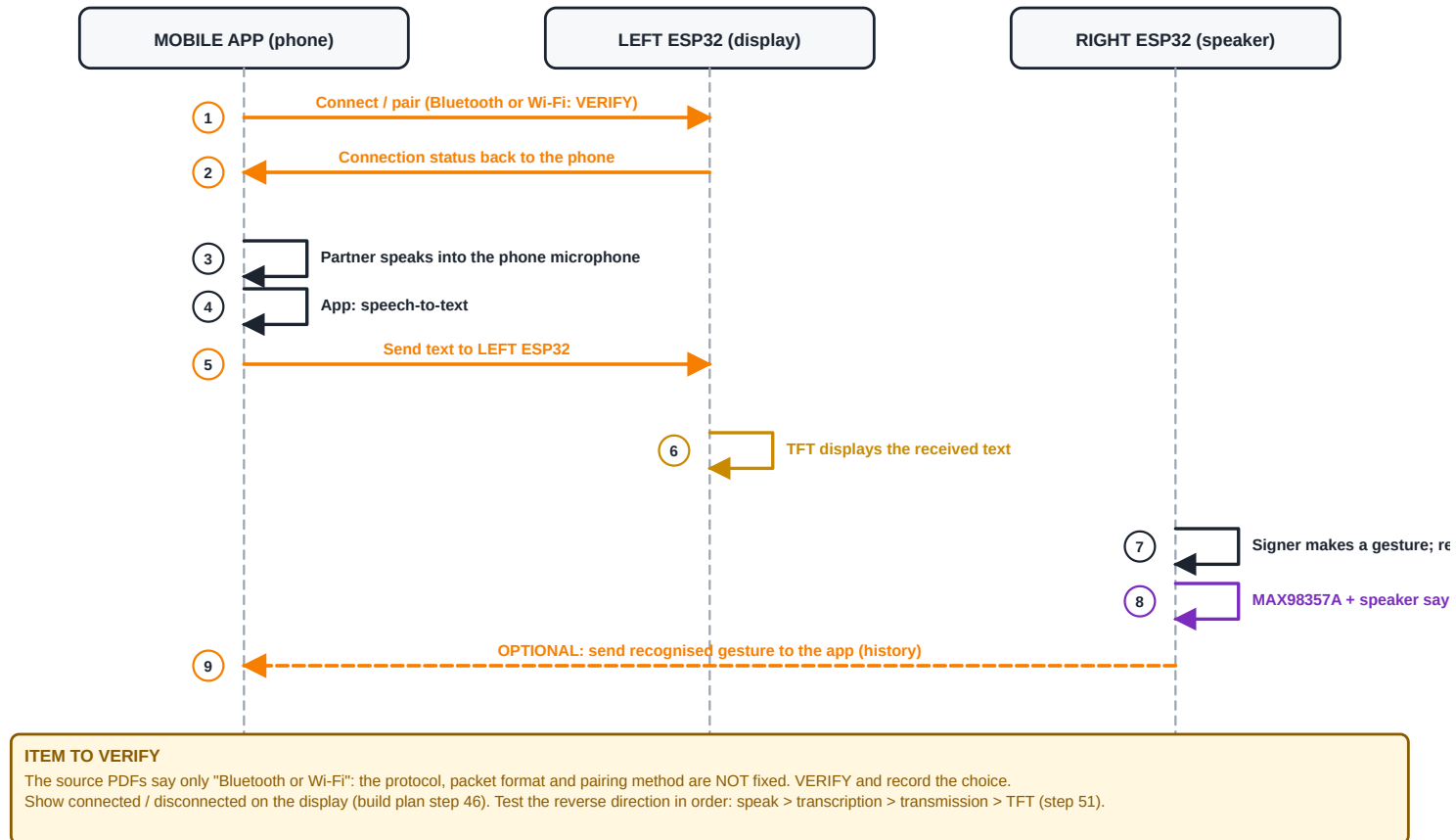
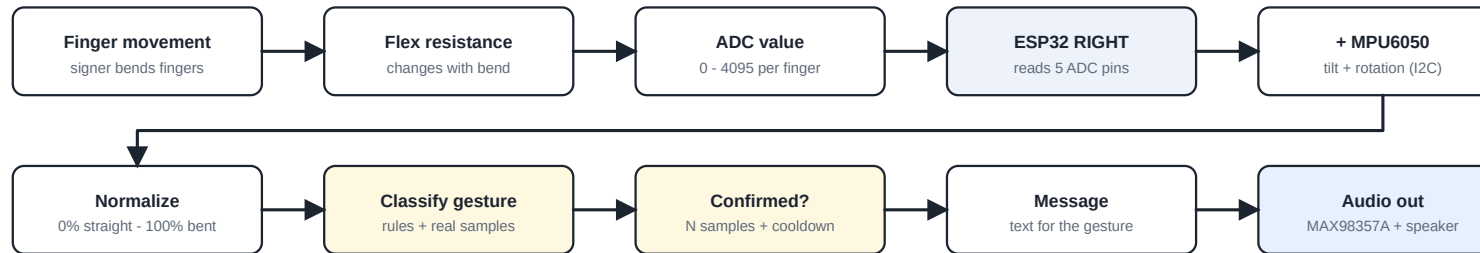


Diagram 11. Message sequence between the phone and the two ESP32 boards.

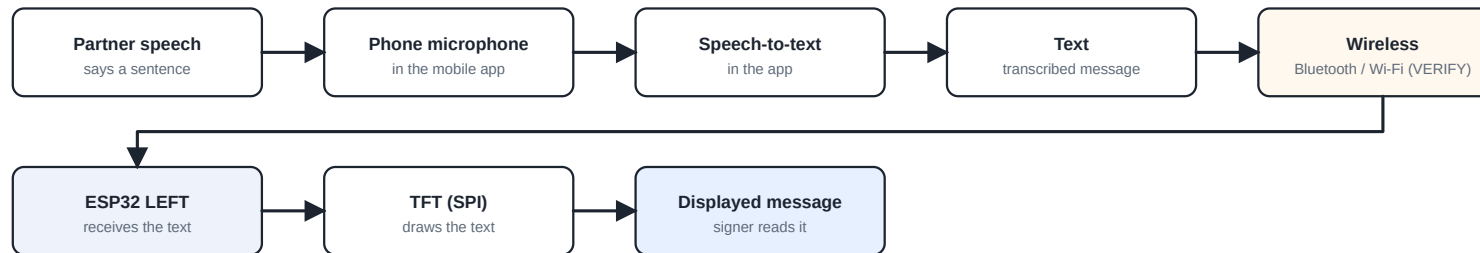
17 Complete data flow

DIAGRAM 12: COMPLETE DATA-FLOW DIAGRAM

FLOW A: sign language to voice (RIGHT glove)



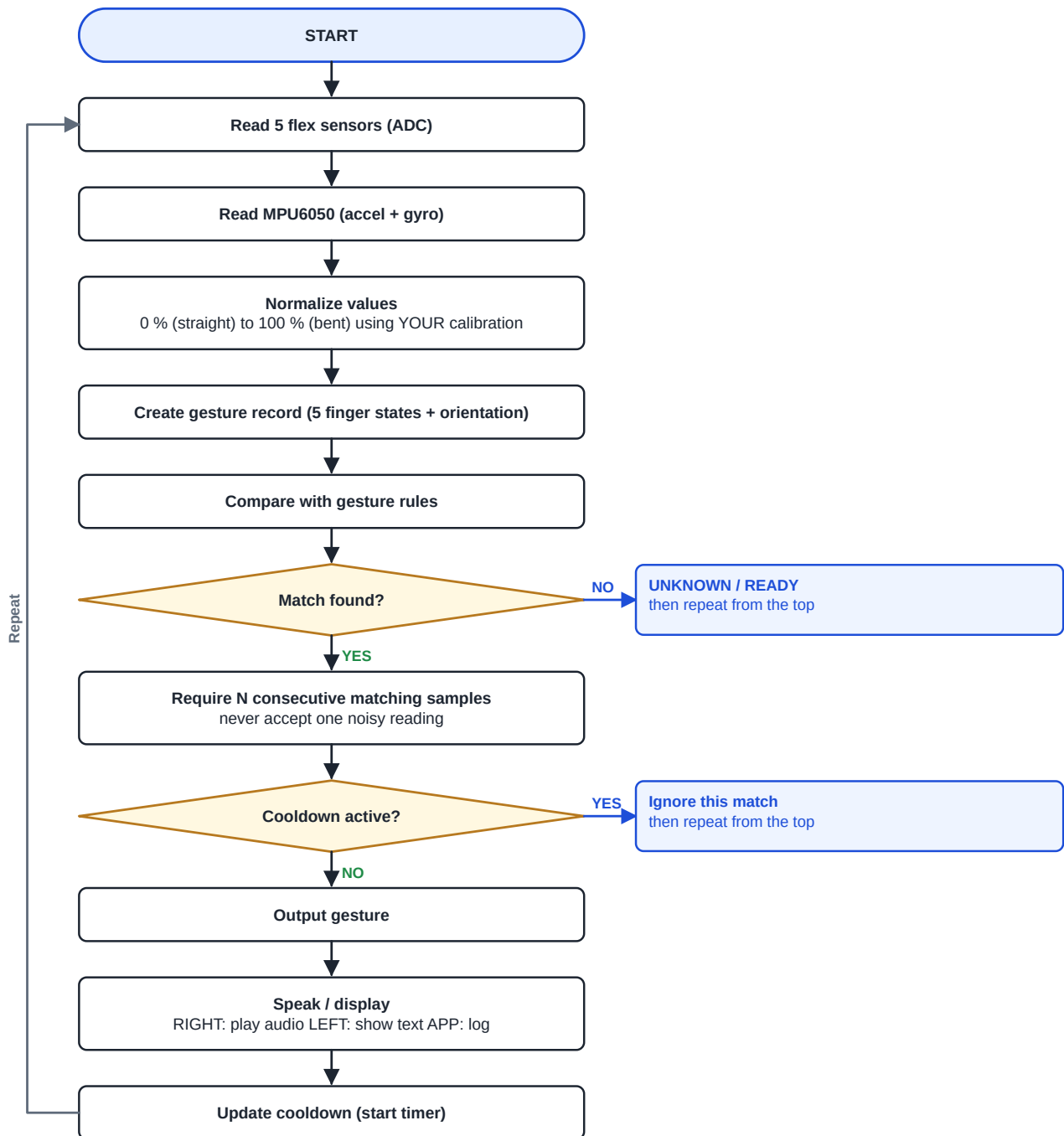
FLOW B: partner speech to text (LEFT glove)



Both flows run on separate hardware. Flow A never needs the phone, except for the optional gesture log. Flow B never needs the right glove.

Diagram 12. Flow A: sign to voice. Flow B: speech to text.

GESTURE RECOGNITION FLOWCHART



Rules (build plan steps 39–44)

- Collect real samples: wear the glove and repeat each gesture; keep good and bad attempts.
- Normalize each finger to 0 % (straight) – 100 % (bent) with **your** calibration.
- Accept a gesture only after **N consecutive** matching samples; then apply a **cooldown** (about 1.5 s in the source example).
- Use MPU6050 orientation to separate gestures that have the same finger pattern.
- If nothing matches, show **UNKNOWN / READY**, never a random message.
- Test transitions: moving between two gestures must not trigger the one in between.

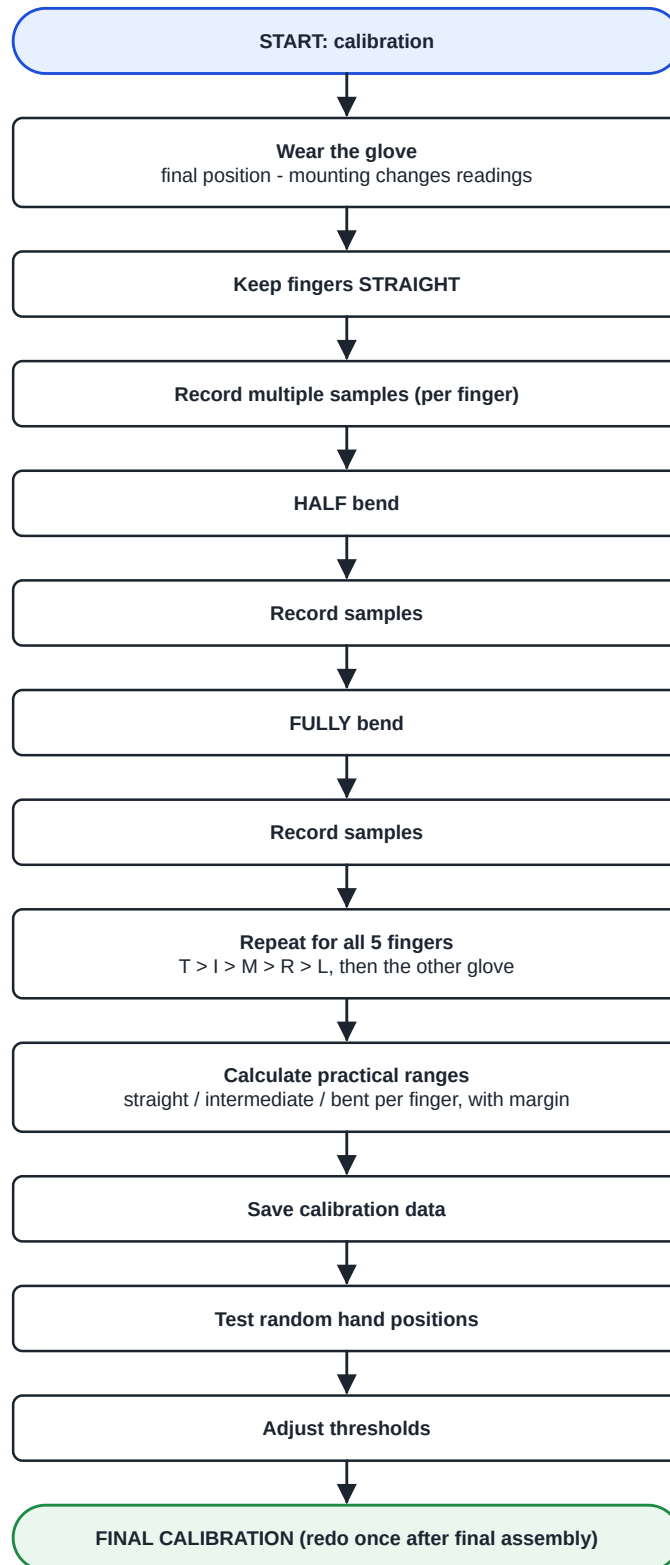
Gesture table template (EXAMPLE only)

Gesture	T	I	M	R	L	Orientation
HELLO	S	S	S	S	S	palm forward
YES	B	B	B	B	B	fist, nod

S = straight, $\frac{1}{2}$ = half, B = bent. **Fill this from YOUR recorded samples.**

CALIBRATION FLOWCHART

Do NOT use example numbers as thresholds. Use your own measured values.



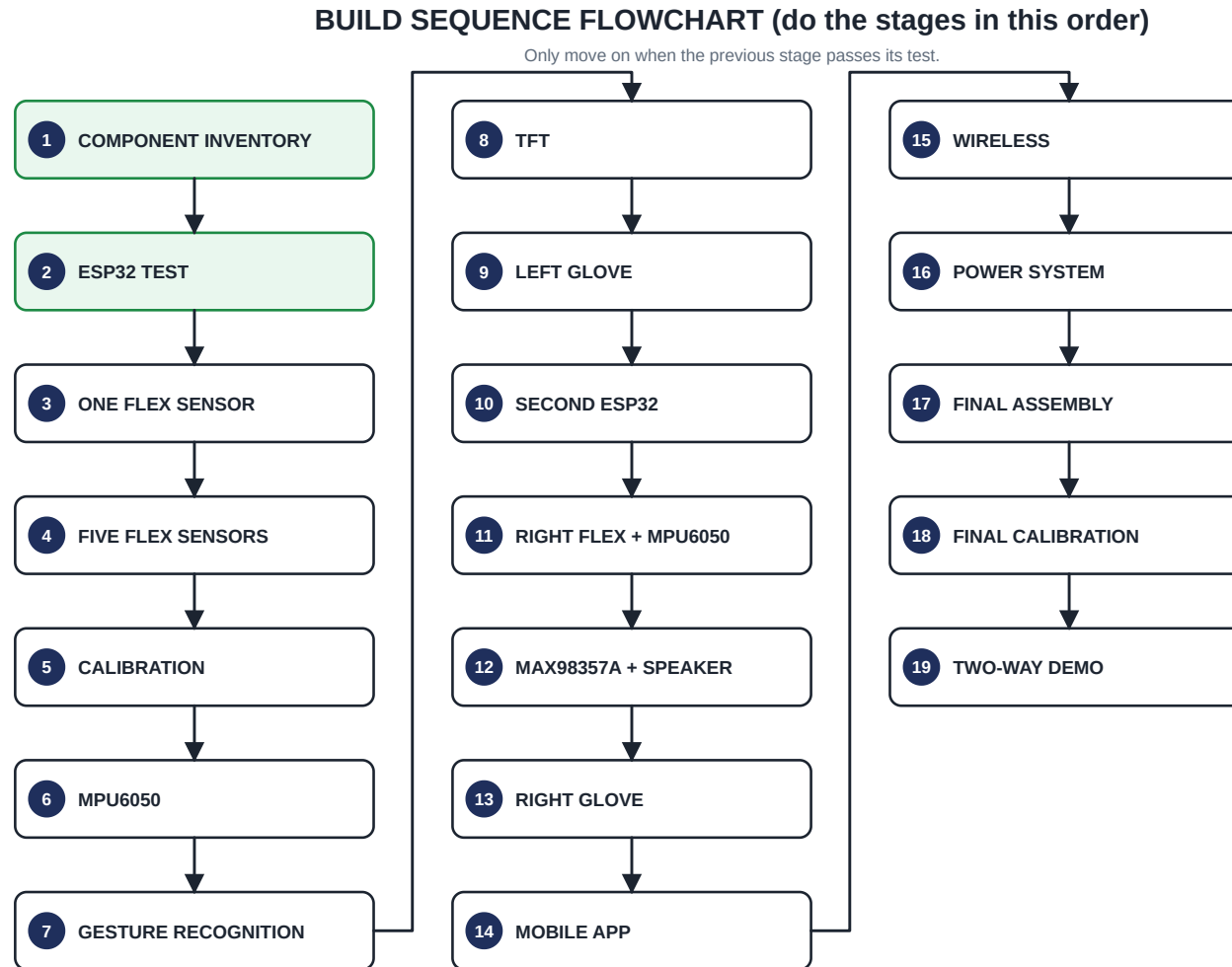
Procedure

- 1. Wear the glove in its final mounting position (mounting changes the readings).
- 2. For each finger: hold **straight** for 3 s and record about 20 samples; then **half bent**; then **fully bent**.
- 3. Record min / max / average for each position and keep a margin between ranges.
- 4. Store the calibration (for example in flash) and reload it at start-up.
- 5. Test random hand positions and adjust thresholds.
- 6. Repeat once more after final assembly.

Do not use example values as final thresholds. Use real measurements from the actual glove.

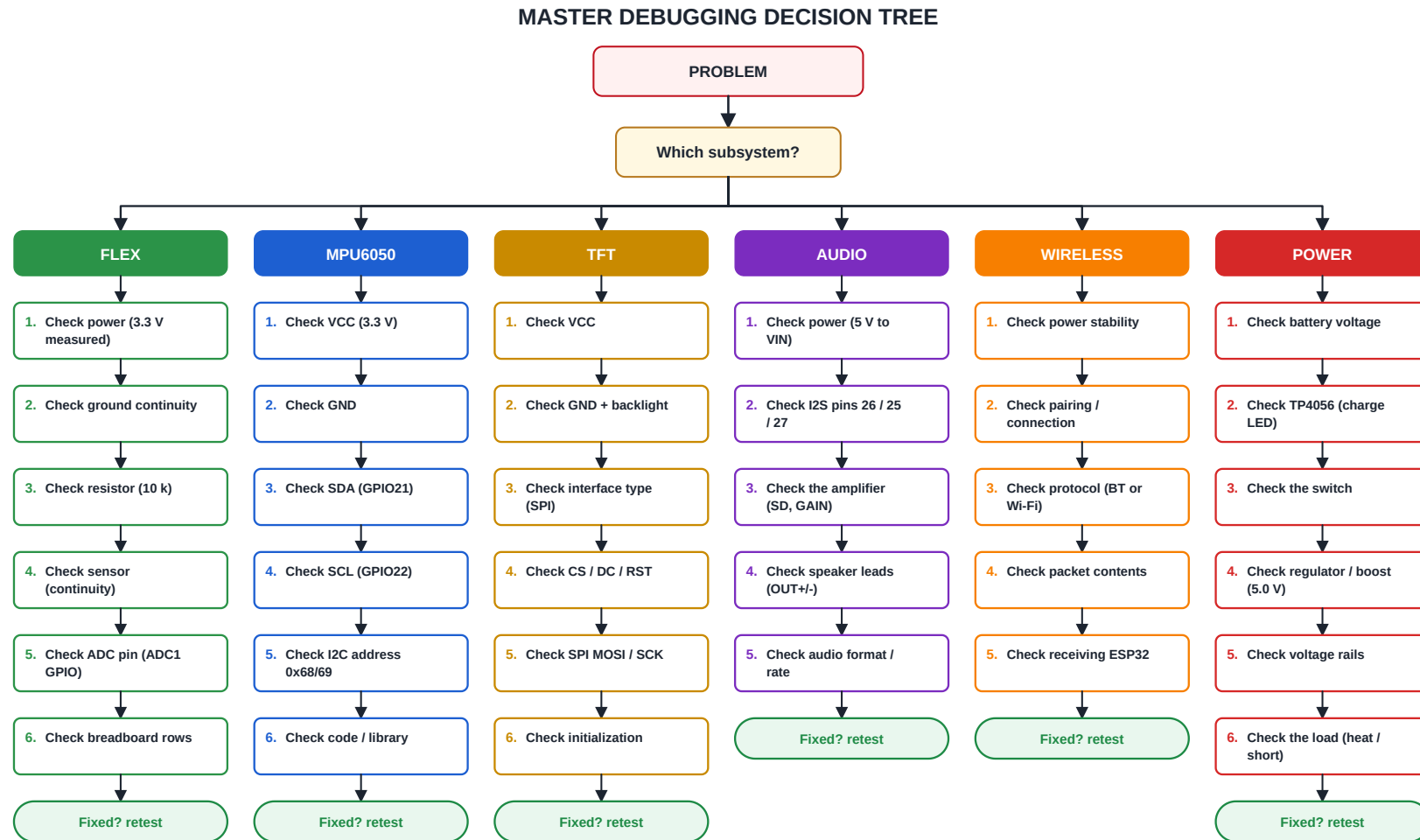
Finger	Straight	Half	Bent
Thumb	_____	_____	_____
Index	_____	_____	_____
Middle	_____	_____	_____
Ring	_____	_____	_____
Little	_____	_____	_____

20 Master flowchart: build sequence



Only move to the next stage when the previous one passes its test.

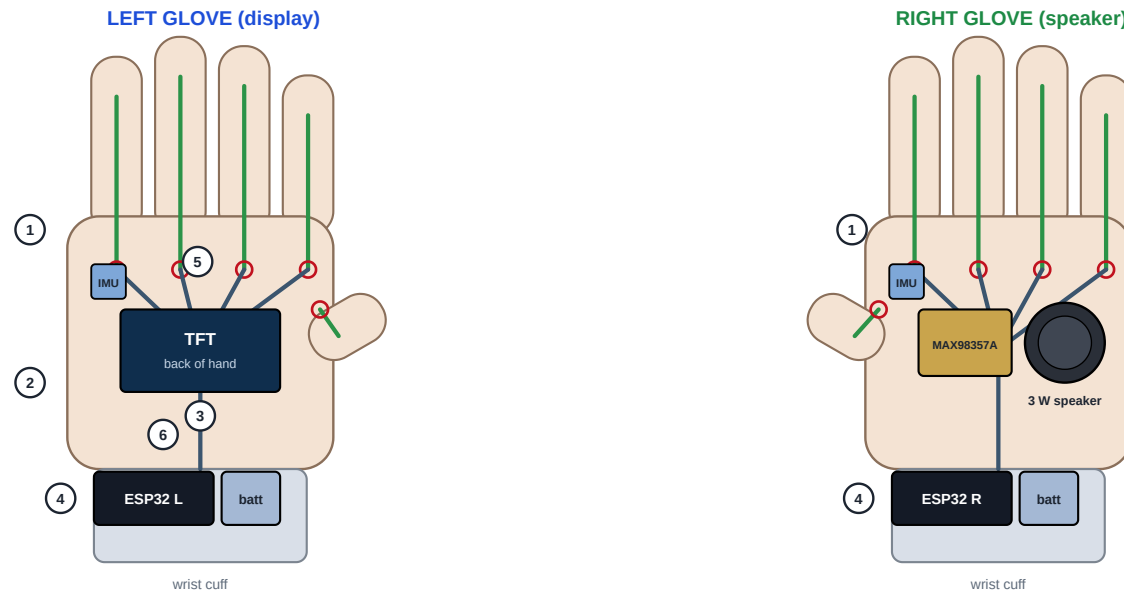
21 Master debugging decision tree



Rule: verify the ELECTRICAL supply (multimeter) before changing any code.

22 Final assembly

FINAL ASSEMBLY DIAGRAM: where every part sits on each glove (back of hand)



- 1 Flex wires run along the back of each finger, never across a knuckle crease.
- 2 Red circles = strain relief: a loop of slack + tape/hot glue at every sensor base.
- 3 Bundle the 5 signal wires (green) into one harness down the centre of the hand.
- 4 ESP32 + battery sit on the wrist cuff: flat, not over the wrist bend.
- 5 Velcro / screws on perfboard: boards stay removable until the final test.
- 6 Heat-shrink or Kapton over every solder joint; no bare metal touches skin.
- 7 MPU6050 lies flat on the back of the hand; axis arrows on the board must be checked.
- 8 Speaker outlet must not be covered by the glove; leads only to amp OUT + / OUT -.

Source: build-plan PDF Figure 16 and steps 53-58. Freeze the wiring only after every test passes (step 68).

Final assembly: freeze wiring only after all tests pass; keep boards removable until then.

Wiring checklist (before power)

- ☐ Power is disconnected while wiring (USB out, battery out).
- ☐ Left and right boards are labelled LEFT / RIGHT; sensors labelled T I M R L.
- ☐ Every flex sensor has its own 10 kΩ resistor and its own ADC wire; no two ADC nodes are joined.
- ☐ Each flex sensor: two tabs in two different rows (breadboard) or on two separate pads (perfboard).
- ☐ Every divider resistor has a GND path (continuity checked).
- ☐ Every sensor has a 3.3 V path (3.3 V measured at its supply row).
- ☐ ADC wires go to GPIO36, 39, 34, 35, 32 only (ADC1).
- ☐ MPU6050: VCC 3.3 V, GND, SDA to GPIO21, SCL to GPIO22.
- ☐ TFT pins compared with the actual module silkscreen; backlight supply verified.
- ☐ MAX98357A pins compared with the actual board; speaker on OUT + / OUT - only, **not to GND**.
- ☐ Battery and TP4056 polarity checked; boost set to 5.0 V before connecting the ESP32.
- ☐ All modules share one GND; no 5 V on any 3.3 V pin.
- ☐ LEFT and RIGHT gloves share no wire.
- ☐ Solder joints insulated; strain relief at every sensor base.

Final test checklist (in this order)

- ☐ ESP32 boots, sketch uploads, Serial Monitor at 115200 (COM port noted).
- ☐ One flex channel: 3.3 V present, reading changes when bent (record straight / half / bent).
- ☐ All five flex channels: only the bent finger changes.
- ☐ Calibration ranges recorded for all 10 sensors.
- ☐ Both MPU6050: I2C scan shows 0x68; accel and gyro respond.
- ☐ TFT shows HASTH VANI / READY / test message.
- ☐ LEFT glove alone: 5 flex + MPU6050 + TFT together.
- ☐ Amplifier plays a known tone; speaker clear and not hot.
- ☐ RIGHT glove alone: 5 flex + MPU6050 + amp + speaker together.
- ☐ Gesture accuracy tested (every gesture, several times, correct vs incorrect recorded).
- ☐ False triggers tested (natural movement, no unwanted message).
- ☐ Response time measured.
- ☐ Wireless range tested for the demo distance.
- ☐ Restart behaviour: both gloves return to a usable state.
- ☐ Battery-only operation stable; charging checked.
- ☐ Full two-way demo: gesture → voice, and speech → text on the TFT.

24 Final validation report

Automatic checks were run by program on the pin map and on the breadboard layout data that draws Diagrams 13–16 (a union-find netlist over breadboard strips and rails). Two deliberate faults were injected to prove the checker can fail.

Result	Check	Detail
PASS	LEFT ESP32: no GPIO is assigned to two peripherals	13 assignments, 13 unique GPIOs
PASS	LEFT ESP32: flex sensors use ADC1 pins only (work with Wi-Fi/BT on)	GPIO 36, 39, 34, 35, 32
PASS	LEFT ESP32: input-only pins (34,35,36,39) are used only as inputs	flex = ADC inputs
PASS	LEFT ESP32: no GPIO 6-11 (flash pins) used	none of 6..11
PASS	LEFT ESP32: strapping pins used are flagged for the user	strapping pins used: GPIO15, GPIO2 (flagged in the TFT section)
PASS	RIGHT ESP32: no GPIO is assigned to two peripherals	10 assignments, 10 unique GPIOs
PASS	RIGHT ESP32: flex sensors use ADC1 pins only (work with Wi-Fi/BT on)	GPIO 36, 39, 34, 35, 32
PASS	RIGHT ESP32: input-only pins (34,35,36,39) are used only as inputs	flex = ADC inputs
PASS	RIGHT ESP32: no GPIO 6-11 (flash pins) used	none of 6..11
PASS	RIGHT ESP32: strapping pins used are flagged for the user	strapping pins used: none (flagged in the TFT section)
PASS	I2C SDA/SCL identical on both gloves (GPIO21/GPIO22)	GPIO21 / GPIO22
PASS	SPI pins (TFT) are the ESP32 VSPI pins 23 / 18 / 19	MOSI23 SCK18 MISO19
PASS	I2S pins are free outputs, not shared with ADC/I2C/SPI	BCLK26 LRC25 DIN27
PASS	Breadboard layout (1 sensor): all 10 netlist checks pass	union-find over strips and rails
PASS	Breadboard layout (5 sensors): all 34 netlist checks pass	union-find over strips and rails
PASS	Mutation test: the checker DETECTS "both tabs in one row"	4 checks fail as they should
PASS	Mutation test: the checker DETECTS "no 3.3 V link"	1 checks fail as they should

Checks reviewed by inspection (not computed)

Check	Note
Every voltage-divider resistor has a GND path	Checked by the breadboard netlist program (above) and visible in Diagrams 2, 3, 7, 8.
Speaker is not connected to GND	By construction in Diagrams 6 and 8 (only OUT + / OUT - are drawn) and marked with a warning box; verify physically.
Battery and TP4056 polarity	Diagram 9 draws B+ to battery +, B- to battery -; verify on the real modules.
No unsupported voltage on a component	TFT/MPU on 3V3; amplifier on 5 V (its datasheet range allows it); ESP32 VIN on 5 V from the boost. Verify each module.
LEFT and RIGHT not merged	Diagram 10: the only link is the phone.
Permanent wiring vs actual module labels	NOT possible here: marked VERIFY ON ACTUAL MODULE for the TFT, MAX98357A, MPU6050 and power modules.

This report checks the *documents*. It cannot see your physical board: the multimeter tests in Section 9 do that.

25 Deliverables and files

#	Deliverable	Where
1	Complete system architecture diagram	Diagram 1 (01_system_architecture.svg)
2	Complete wiring schematic	Diagram 10 + Diagrams 3, 7, 8
3	Left glove schematic	Diagram 7
4	Right glove schematic	Diagram 8
5	Five-flex breadboard diagram	Diagram 14
6	One-flex breadboard diagram	Diagram 13
7	MPU6050 wiring diagram	Diagram 4
8	TFT wiring diagram	Diagram 5
9	MAX98357A + speaker diagram	Diagram 6
10	Battery / TP4056 power diagram	Diagram 9
11	Mobile app communication diagram	Diagrams 11 and 12
12	Gesture-recognition flowchart	Flowchart 20
13	Calibration flowchart	Flowchart 21
14	Flex-sensor troubleshooting flowchart	Flowchart 17
15	Complete debugging decision tree	Flowchart 23
16	Final assembly diagram	Diagram 24
17	Pin mapping table	Section 5
18	Wiring checklist	Section 23
19	Testing checklist	Section 23
20	Professional PDF hardware manual	this document

Individual diagrams are also supplied as separate SVG and PNG files in the `diagrams` folder.